

Machine Learning: Guidelines and Evaluation of Classification Algorithms

1 Guidelines for Machine Learning Algorithms

1. Understand the Problem and the Data

Before applying any machine learning algorithm, it is essential to understand both the problem you are solving and the nature of the data available.

- **Clearly define the task:** Identify whether the problem is *classification*, *regression*, or *clustering*.

Suppose you have student data with features such as study hours and marks obtained.

- If the task is to predict whether a student will **pass or fail**, it is a **classification** problem.
- If the task is to predict the **exact marks out of 100**, it is a **regression** problem.
- If the task is to group students into **high, medium, low performers** without predefined labels, it is a **clustering** problem.
- **Analyze data characteristics:** Check for data size, dimensionality, missing values, and class imbalance. Consider the following dataset of 6 students:

Student	Study Hours	Pass/Fail
<i>S1</i>	2	<i>Fail</i>
<i>S2</i>	4	<i>Fail</i>
<i>S3</i>	5	<i>Pass</i>
<i>S4</i>	6	<i>Pass</i>
<i>S5</i>	?	<i>Pass</i>
<i>S6</i>	7	<i>Pass</i>

- **Missing Values:** S5 has missing study hours.
- **Class Imbalance:** Out of 6 students, 4 are Pass and 2 are Fail (unequal distribution).
- **Dimensionality:** Only 1 feature (Study Hours). In real problems, datasets may have 10s or 100s of features.

- **Use Exploratory Data Analysis (EDA):** Perform summary statistics, visualization, and correlation checks to understand data patterns. From the above dataset:
 - **Mean Study Hours** (ignoring missing): $\frac{2+4+5+6+7}{5} = 4.8$ hours.
 - Students who studied less than 4 hours mostly failed, while those with more than 5 hours mostly passed.
 - Outlier check: If one student had studied 20 hours (very high compared to others), it would be an outlier.

2. Data Preprocessing

Effective preprocessing improves both model performance and reliability. Below are common steps with simple numerical examples. **Always compute any statistics (e.g., means, standard deviations) on the *training set only*** to avoid leakage.

- **Handle missing values**

1. *Numeric features: mean/median imputation*

Consider a numeric feature x with values: [2, 4, NA, 6, 100].

- **Mean (sensitive to outliers):**

$$\mu = \frac{2 + 4 + 6 + 100}{4} = \frac{112}{4} = 28$$

Impute NA with 28 $\Rightarrow$ [2, 4, 28, 6, 100].

- **Median (robust to outliers):** Data (sorted, ignoring NA): [2, 4, 6, 100]; median = $\frac{4+6}{2} = 5$. Impute NA with 5 $\Rightarrow$ [2, 4, 5, 6, 100].

Tip: If the feature has strong outliers, prefer median or a robust approach.

2. *Categorical features: mode or “Unknown”*

Suppose **City** = [A, B, B, C, NA]. Mode is B. Impute NA with B: [A, B, B, C, B]. Alternatively, impute with a new label **Unknown** to explicitly mark missingness.

3. *Row/column deletion (use sparingly)*

If a row has many missing fields, or a column is $> 60\%$ missing and not critical, deletion can be justified. Otherwise prefer imputation.

4. *Missing-indicator features*

Add a binary flag (e.g., m_x) where $m_x = 1$ if x was missing, else 0. This lets models learn that “missingness” itself may carry signal.

- **Encode categorical variables (One-Hot, Label, Ordinal)**

1. *Label encoding (for tree models or genuine ordinals)*

Color = [Red, Blue, Blue, Green]. Map: **Red** $\rightarrow$ 0, **Blue** $\rightarrow$ 1, **Green** $\rightarrow$ 2.

Encoded = [0, 1, 1, 2]. *Note:* Linear models and distance-based models may incorrectly treat $2 > 1 > 0$ as meaningful distances. Use One-Hot there.

2. *One-Hot encoding (safe for nominal categories)*

Color	Red	Blue	Green
Red	1	0	0
Blue	0	1	0
Blue	0	1	0
Green	0	0	1

Tip: To avoid multicollinearity in linear models, you may drop one dummy (e.g., drop the Red column).

3. *Ordinal encoding (when order matters)*

Education = [HighSchool, Bachelors, Masters] Encode with order:
HighSchool $\rightarrow$ 1, Bachelors $\rightarrow$ 2, Masters $\rightarrow$ 3.

4. *High-cardinality categories*

Many unique values (e.g., ZIP codes) can create huge One-Hot vectors. Consider hashing tricks or learned embeddings; be careful with target encoding (risk of leakage).

• **Normalize / scale features**

1. *Standardization (z-score)*

$$z = \frac{x - \mu}{\sigma}$$

Example (train-only fit): Given training values $x = [2, 4, 5, 6, 7]$,

$$\mu = \frac{2 + 4 + 5 + 6 + 7}{5} = \frac{24}{5} = 4.8$$

Using sample standard deviation,

$$\sigma = \sqrt{\frac{\sum (x - \mu)^2}{n - 1}} = \sqrt{\frac{(2-4.8)^2 + (4-4.8)^2 + (5-4.8)^2 + (6-4.8)^2 + (7-4.8)^2}{4}}$$

Then

$$z(x) \approx [-1.46, -0.42, 0.10, 0.62, 1.15].$$

For a *test* value $x_{\text{test}} = 5.5$, apply the *same* μ, σ from train:

$$z_{\text{test}} = \frac{5.5 - 4.8}{1.923} \approx 0.36.$$

2. *Min-Max scaling* to $[0, 1]$

$$x' = \frac{x - \min(x)}{\max(x) - \min(x)}.$$

With $x = [2, 4, 5, 6, 7]$, $\min = 2$, $\max = 7$. For $x = 6$,

$$x' = \frac{6 - 2}{7 - 2} = \frac{4}{5} = 0.8.$$

3. *Robust scaling* (for outliers)

$$x' = \frac{x - \text{median}}{\text{IQR}} \quad (\text{IQR} = Q3 - Q1).$$

Prefer when data have heavy tails/outliers.

4. *When to scale*: Scaling is crucial for distance- and gradient-based models (KNN, K-means, SVM, Logistic/Linear Regression, Neural Nets). Tree-based methods are less sensitive.

- **Split the dataset into training, validation, and test sets**

1. *Typical ratios*: 60/20/20 or 70/15/15. Ensure the **test set is held out** until final evaluation.
2. *Stratified splits (for classification)*:
Suppose $N = 10$ with labels $y = [1, 0, 1, 0, 0, 1, 0, 0, 0, 1]$ (positives= 4, negatives= 6). A 60/20/20 *stratified* split keeps class proportions similar in each split (e.g., Train $\approx$ 2–3 positives, 3–4 negatives; Val $\approx$ 1 pos/1 neg; Test $\approx$ 1 pos/1 neg).
3. *No leakage rule*: Fit imputers/scalers/encoders only on **train**, then apply to validation/test. If using cross-validation, fit them *inside each fold*.

3. Model Selection

Choosing the right algorithm depends on the type of task, the structure of the dataset, and the balance between **interpretability** (how easily humans can understand the model) and **predictive power** (how well the model generalizes).

- **Choose models depending on data structure and task complexity:**

- If the data is **small and simple** with a linear relationship, use simpler models like Logistic Regression or Linear Regression.
- If the data is **non-linear** or **large and complex**, use more flexible models such as Random Forests, Gradient Boosted Trees, or Neural Networks.
- For **unsupervised learning** tasks, clustering or dimensionality reduction methods are more suitable.

- **Interpretability vs. Predictive Power:**

Sometimes, simpler models are chosen even if they are slightly less accurate because they are easier to interpret. In critical domains such as healthcare or finance, interpretability is often more important. In contrast, for applications like image classification, predictive accuracy may be prioritized.

Example 1. Suppose we want to predict whether a student will pass based on study hours.

- A **Logistic Regression** model may predict:

$$P(\text{Pass}) = \frac{1}{1 + e^{-(0.5 \cdot \text{Hours} - 2)}}$$

This model is highly interpretable because we can directly see the effect of study hours.

- A **Random Forest Classifier** might give slightly higher accuracy by capturing more complex patterns, but it is less transparent (harder to explain how each decision is made).

Examples of Algorithms

Task	Typical Algorithms
Classification	Logistic Regression (linear, interpretable), SVM (handles complex boundaries), KNN (instance-based), Decision Tree (rule-based), Naive Bayes (probabilistic)
Regression	Linear Regression (simple), SVR (non-linear), Random Forest Regressor (robust)
Clustering	K-Means (centroid-based), DBSCAN (density-based), Agglomerative Clustering (hierarchical)
Dim. Reduction	PCA (linear), t-SNE (non-linear, for visualization), UMAP (manifold learning)

Numerical Illustration

Consider the dataset:

Student	Hours Studied	Pass/Fail
S1	2	<i>Fail</i>
S2	4	<i>Fail</i>
S3	5	<i>Pass</i>
S4	7	<i>Pass</i>

Suppose we want to predict the outcome for a new student who studied **6 hours**. Different algorithms approach this prediction differently:

- **Logistic Regression:** Logistic regression fits a sigmoid (S-shaped) curve to estimate probabilities. For example, a fitted model might be:

$$P(\text{Pass}) = \frac{1}{1 + e^{-(1.2 \cdot \text{Hours} - 5)}}$$

Substituting Hours = 6:

$$P(\text{Pass}) = \frac{1}{1 + e^{-(1.2 \cdot 6 - 5)}} \approx 0.77$$

Since $P > 0.5$, the prediction is **Pass**. The advantage here is interpretability: we can directly see how the number of hours influences the probability of passing.

- **K-Nearest Neighbors (KNN, $k = 3$):** KNN predicts based on the closest data points.

Distances: $d(6, 2) = 4$, $d(6, 4) = 2$, $d(6, 5) = 1$, $d(6, 7) = 1$

The 3 nearest neighbors are at 5 (Pass), 7 (Pass), and 4 (Fail). Majority = 2 Pass vs. 1 Fail $\Rightarrow$ prediction = **Pass**. This model is simple but can change with k ; for $k = 1$, the nearest neighbor is 5 (Pass).

- **Decision Tree:** A decision tree splits the data into regions using simple rules. A possible learned rule:

If Hours $< 4.5 \Rightarrow$ Fail, Else Pass.

For Hours = 6, the condition is “Else Pass,” so prediction = **Pass**. The model is interpretable because decisions can be traced as simple if-else rules.

- **Comparison:**
 - Logistic Regression: Uses a smooth probability boundary; interpretable coefficients.
 - KNN: Instance-based, depends on neighbors; flexible but less interpretable.
 - Decision Tree: Rule-based, highly interpretable, can overfit on small datasets.

Despite using different approaches, all three algorithms predict **Pass** for 6 hours of study.

4. Training the Model

Training is the process of fitting a model to the training data so that it can make predictions on unseen data. To ensure reliability and avoid overfitting, certain techniques are essential.

- **Use k -fold cross-validation for reliable generalization**

Instead of training and testing on a single split, k -fold cross-validation divides the training data into k subsets (folds). The model is trained on $k - 1$ folds and validated on the remaining fold, repeated k times. The average performance gives a more robust estimate of generalization.

Example 2. *Suppose we have 6 data points: $[1, 2, 3, 4, 5, 6]$ and use 3-fold CV.*

- *Fold 1: Train = $[3, 4, 5, 6]$, Validation = $[1, 2]$*
- *Fold 2: Train = $[1, 2, 5, 6]$, Validation = $[3, 4]$*
- *Fold 3: Train = $[1, 2, 3, 4]$, Validation = $[5, 6]$*

Each point gets to be in validation once. The average accuracy across folds estimates the model’s performance more reliably than a single split.

- **Prevent overfitting using:**

Overfitting happens when a model performs well on training data but poorly on unseen data. Techniques to reduce overfitting include:

- **Regularization: L1 (Lasso) and L2 (Ridge)** Add a penalty term to the loss function.

$$\text{Loss}_{\text{Ridge}} = \text{MSE} + \lambda \sum_j w_j^2$$

$$\text{Loss}_{\text{Lasso}} = \text{MSE} + \lambda \sum_j |w_j|$$

Example 3. Suppose Linear Regression finds $y = 5x_1 + 4x_2 + 0.1x_3$.

- * With Ridge (L2), large weights (e.g., 5 and 4) are reduced.
- * With Lasso (L1), irrelevant weights (e.g., 0.1) may shrink exactly to 0, effectively removing x_3 .

- **Early stopping** (for iterative algorithms) Stop training when validation accuracy stops improving.

Example 4. In neural network training, validation accuracy might increase until epoch 15, then flatten or decrease. Stopping at epoch 15 (instead of continuing to 50) prevents overfitting.

- **Pruning (Decision Trees)** Limit depth or remove branches that provide little improvement.

Example 5. A decision tree trained without pruning may split until each leaf has 1 sample (overfit). By restricting depth to 3, the tree generalizes better.

- **Dropout (Neural Networks)** Randomly drop neurons during training to prevent reliance on specific nodes.

Example 6. In a neural network with 100 neurons, applying dropout of 20% randomly removes about 20 neurons each iteration. This forces the network to learn robust, distributed representations.

5. Hyperparameter Tuning

Machine learning models have parameters learned from data (e.g., weights in regression, splits in trees) and **hyperparameters** set before training (e.g., learning rate, regularization strength, number of neighbors). Proper tuning of hyperparameters can significantly improve performance.

- **Use Grid Search or Random Search on validation data**

- **Grid Search:** Systematically tries all combinations of hyperparameters. Example: For k -Nearest Neighbors (KNN), suppose we want to test $k \in \{3, 5\}$ and distance metric $\in \{\text{Euclidean}, \text{Manhattan}\}$. Grid search will evaluate all $2 \times 2 = 4$ combinations:

$\{(k = 3, \text{Euclidean}), (k = 3, \text{Manhattan}), (k = 5, \text{Euclidean}), (k = 5, \text{Manhattan})\}$.

- **Random Search:** Samples hyperparameter combinations at random within specified ranges. Example: For a Decision Tree, choose

$$\text{max_depth} \sim \text{Uniform}(2, 20), \quad \text{min_samples_split} \sim \text{Uniform}(2, 10).$$

If we randomly pick (depth = 7, split = 4), we train and evaluate the tree with these values. Random search is more efficient when the hyperparameter space is large.

- **Random Search (Support Vector Machine):** Randomly samples hyperparameters for an SVM classifier.

$$C \sim \text{LogUniform}(10^{-3}, 10^3), \quad \gamma \sim \text{LogUniform}(10^{-4}, 10^1)$$

If we sample ($C = 15.2$, $\gamma = 0.03$), we train and test the SVM with these parameters.

- **Random Search (Neural Network):** Randomly samples hyperparameters for a feed-forward neural network.

$$\text{learning_rate} \sim \text{LogUniform}(10^{-5}, 10^{-1}), \quad \text{hidden_units} \sim \text{Uniform}\{32, 64, 128, 256\}$$

If we sample (lr = 0.0012, units = 128), we train the model with these settings.

- **Random Search (k-Nearest Neighbors):** Randomly samples hyperparameters for kNN.

$$k \sim \text{Uniform}\{1, 2, \dots, 30\}, \quad \text{weights} \in \{\text{uniform}, \text{distance}\}$$

If we pick ($k = 17$, weights = distance), we evaluate kNN using these values.

- **Random Search (Random Forest):**

$$\text{n_estimators} \sim \text{Uniform}(50, 500), \quad \text{max_features} \sim \text{Uniform}\{1, 2, \dots, 10\}$$

If we pick (estimators = 320, features = 7), we train the forest with these parameters.

- **Evaluate performance on hold-out test set**

After selecting the best hyperparameters based on validation data, retrain the model using the full training + validation set, and finally evaluate on the unseen test set.

Numerical Example: Suppose for KNN:

- Validation accuracy with $k = 3$: 82%
- Validation accuracy with $k = 5$: 85%

We select $k = 5$. Now retrain with $k = 5$ using all training data, and test on the held-out test set. If the test accuracy = 83%, this is the unbiased estimate of performance.

- **Important Note:**

Hyperparameter tuning should never use the test set directly, as this leads to *data leakage* and overly optimistic results. The test set must remain untouched until the very end.

6. Feature Engineering

Feature engineering refers to creating, selecting, and transforming features to improve the performance of machine learning models. Good features often matter more than complex algorithms.

- **Use domain knowledge to create meaningful features**

Understanding the context of the problem can help design better features.

Example: In predicting house prices, raw features may include area, number of rooms, and year built. Using domain knowledge, we can create new features such as:

- **Price per Square Foot** = $\frac{\text{Price}}{\text{Area}}$
- **House Age** = Current Year – Year Built

These derived features can capture more meaningful patterns.

- **Feature selection**

Not all features improve the model. Irrelevant or redundant features may increase complexity and reduce accuracy. Below are common selection techniques:

- **Correlation analysis**

Compute correlation coefficients between features and target. Highly correlated features are more informative. **Example:** Suppose the correlation with target (exam score) is:

Study Hours: 0.85, Sleep Hours: 0.60, Favorite Color: 0.01

Clearly, **Favorite Color** can be removed since it is uncorrelated.

- **Recursive Feature Elimination (RFE)**

Start with all features, train a model, rank features by importance, and remove the least important one iteratively until only the best remain.

Example: With 5 features {A,B,C,D,E}, suppose after training a linear model, feature importances are:

[$A = 0.4$, $B = 0.3$, $C = 0.2$, $D = 0.05$, $E = 0.05$]. RFE may first remove D and E, and then re-train on A,B,C.

- **Tree-based feature importance**

Decision trees and ensembles (Random Forest, XGBoost) can rank features based on how much they reduce impurity. **Example:** A Random Forest model predicts employee attrition. Feature importance scores may be:

Years at Company: 0.40, Monthly Salary: 0.35, Commute Distance: 0.15, Employee ID

Here, **Employee ID** is irrelevant and can be dropped.

7. Model Interpretation

Model interpretation helps to understand *why* a machine learning model makes certain predictions, improving transparency and trust.

- **LIME (Local Interpretable Model-agnostic Explanations):** Approximates the complex model with an interpretable surrogate model (e.g., linear regression) around a specific prediction to highlight important features locally.
- **Permutation Importance:** Evaluates feature importance by randomly shuffling each feature and measuring the change in model performance, giving a global sense of which features the model relies on most.

Summary: These methods provide both *local interpretability* (e.g., SHAP, LIME for individual predictions) and *global interpretability* (e.g., permutation importance for overall feature influence).

8. Model Deployment

- Serialize model using `joblib`, `pickle`, or `ONNX`.
- Deploy via REST APIs or cloud platforms.
- Monitor performance for concept drift.

Statistical Significance test in Machine Learning

Concept, Application and Numerical Example

1 Statistical Significance

Statistical significance testing is used to determine whether an observed difference between two results is likely to be a real difference or could have occurred due to random variation.

For example, when comparing two machine learning models, suppose Model A achieves an accuracy of 92% and Model B achieves an accuracy of 89%. The observed difference is

$$92\% - 89\% = 3\%.$$

A statistical significance test helps determine whether this 3% difference is statistically meaningful. The statistical test begins with two hypotheses:

H_0 : There is no significant difference between the two models.

H_1 : There is a significant difference between the two models.

A commonly used significance level is

$$\alpha = 0.05.$$

This is called the **5% significance level**. The corresponding confidence level is

$$1 - \alpha = 1 - 0.05 = 0.95 = 95\%.$$

Thus,

$5\% \text{ significance level} \iff 95\% \text{ confidence level}$

The statistical decision is commonly based on the p -value. If

$$p < 0.05,$$

the null hypothesis H_0 is rejected and the difference is considered **statistically significant**.
If

$$p \geq 0.05,$$

the null hypothesis is not rejected, and the difference is considered **not statistically significant**.

1.1 Selection of Statistical Significance Tests

The selection of an appropriate statistical significance test depends on the following factors:

- **Number of models being compared:** The first step is to determine whether two models or three or more models are being compared.

- **Relationship between observations:** The observations should be identified as either independent or paired.

For example, suppose the accuracy of two machine learning models is evaluated on different datasets:

Dataset	D_1	D_2	D_3	D_4	D_5
Model A	91	89	93	90	92
Model B	85	87	88	86	89

If Model A and Model B are evaluated on **different and independent datasets**, the observations are considered independent. In this case, an independent t-test can be considered.

In contrast, suppose both models are evaluated on the **same datasets**:

Dataset	D_1	D_2	D_3	D_4	D_5
Model A	91	89	93	90	92
Model B	88	87	91	88	90

Here, each performance value from Model A has a corresponding performance value from Model B for the same dataset. Therefore, the observations are **paired**.

- **Distribution of the data:**

For paired observations, the distribution of the differences between the two models should be examined before selecting an appropriate statistical test. Consider two machine learning models, Model A and Model B, evaluated on the same set of datasets. Since both models are evaluated on the same datasets, the observations are paired.

The following table shows the performance of the two models:

Table 1: Performance of two models on paired datasets.

Dataset	Model A	Model B	Difference ($A - B$)
D_1	91	94	-3
D_2	88	90	-2
D_3	92	94	-2
D_4	90	91	-1
D_5	94	95	-1
D_6	90	90	0
D_7	93	92	1
D_8	91	90	1
D_9	92	90	2
D_{10}	95	93	2
D_{11}	96	93	3

For each dataset, the paired difference is calculated as

$$d_i = A_i - B_i.$$

Therefore, the set of paired differences is

$$d = [-3, -2, -2, -1, -1, 0, 1, 1, 2, 2, 3].$$

The differences are approximately symmetric around zero. Their frequency distribution is

d	-3	-2	-1	0	1	2	3
f	1	2	2	1	2	2	1

This distribution is reasonably symmetric and can be considered an illustrative example of approximately normally distributed differences.

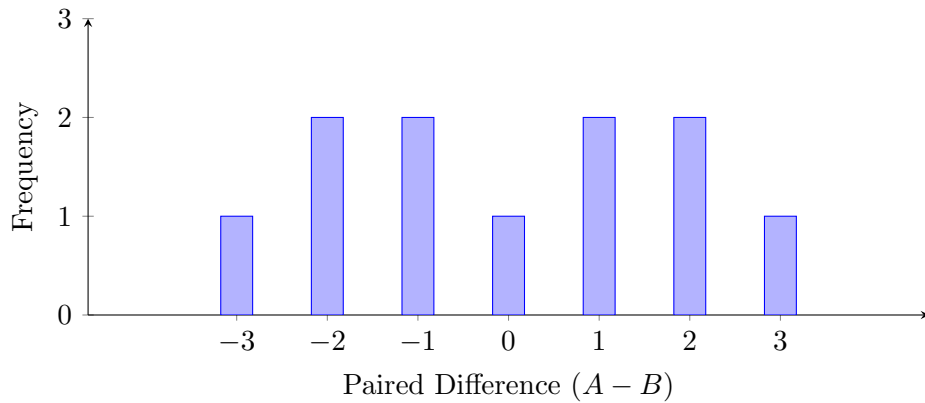


Figure 1: Approximately symmetric distribution of paired differences.

In contrast, consider another set of paired differences:

$$d = [1, 1, 1, 1, 1, 2, 2, 2, 3, 10].$$

The corresponding frequency distribution is

d	1	2	3	10
f	5	3	1	1

In this case, most of the differences are concentrated around small values, while the value 10 produces a long right tail. Therefore, the distribution is strongly right-skewed and cannot be considered approximately normal.

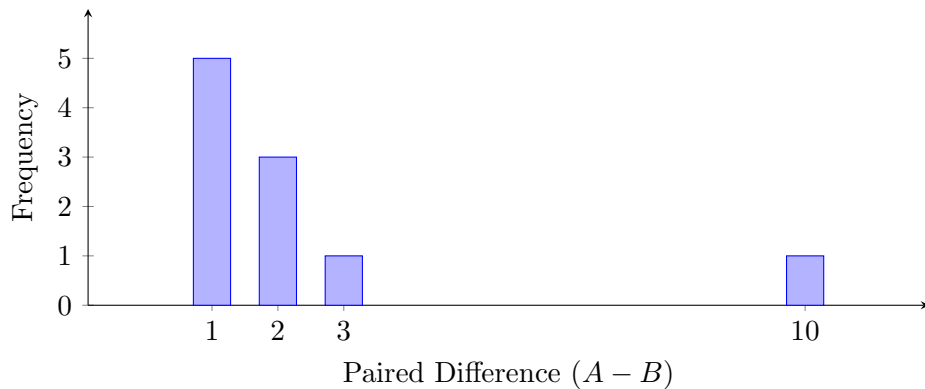


Figure 2: Right-skewed distribution of paired differences.

Thus, the distribution of the paired differences is an important factor in selecting the statistical test. When the paired differences are approximately normally distributed, a paired t-test can be used. When the differences are substantially non-normal, the Wilcoxon signed-rank test can be considered as a non-parametric alternative.

The general selection can therefore be summarized as

Paired observations + approximately normal differences	→ Paired t-test
Paired observations + non-normal differences	→ Wilcoxon signed-rank test

- **Type of comparison:** The statistical test also depends on the type of performance measurements being compared. Continuous performance measures such as accuracy, precision, recall, F1-score, or AUC may require different analysis from categorical predictions such as correct and incorrect classifications.
- **Number of groups and pairing:** For two independent groups, an independent t-test can be used. For two paired groups, a paired t-test or Wilcoxon signed-rank test can be used. For three or more independent groups, one-way ANOVA can be used, whereas the Friedman test can be used for three or more paired groups.
- **Classifier predictions:** When two classifiers are evaluated on the same individual test samples and the outcome is categorical, McNemar's test can be used to compare their predictions.

2 What is an Independent t-Test?

An independent (unpaired) t-test is a statistical hypothesis test used to determine whether the means of two **independent groups** are significantly different from each other.

In machine learning research, it can be used to determine whether the performance of two models is significantly different when the performance measurements come from **independent samples or independent test sets**.

The key requirement is that the observations in one group are not directly related or matched to observations in the other group.

3 When is it Used in Machine Learning?

An independent t-test can be considered in situations such as:

- Comparing the accuracy of two ML models evaluated on two independent test sets.
- Comparing the mean prediction error of two models using independent samples.
- Comparing the performance of two algorithms on independent experimental runs, when the runs are independently generated.
- Comparing the performance of two learning approaches using independent groups of samples.

3.1 When Should It NOT Be Used?

An independent t-test should generally not be used when the observations are paired or related.

For example:

- Model A and Model B are evaluated on the **same test samples**.
- Model A and Model B are evaluated on the **same datasets**.
- The same cross-validation folds are used to obtain paired performance values for two models.

In such cases, the observations are related and a **paired t-test** or an appropriate non-parametric test such as the **Wilcoxon signed-rank test** may be more appropriate.

4 Simple Rule for Machine Learning

Experimental Situation	Appropriate Test
Two independent groups	Independent t-test
Two paired model results	Paired t-test
Three or more independent groups	One-way ANOVA
Three or more paired model results	Friedman test

5 Machine Learning Numerical Example

Consider a binary classification problem in which we want to compare the performance of two machine learning algorithms:

- **Model A:** Support Vector Machine (SVM)
- **Model B:** Logistic Regression

Suppose the researcher evaluates the two models using **independent test samples** and obtains five independent accuracy measurements for each model.

The accuracy values are expressed as percentages.

Model A – SVM

80, 82, 78, 85, 80

Model B – Logistic Regression

70, 72, 68, 75, 70

The observations belonging to Model A and Model B are assumed to come from independent experimental samples.

Therefore, an independent t-test is appropriate.

We use a significance level of

$$\alpha = 0.05.$$

6 Step 1: Formulate the Hypotheses

Let

μ_A = population mean accuracy of Model A

and

μ_B = population mean accuracy of Model B.

The null hypothesis is

$$H_0 : \mu_A = \mu_B$$

which means that there is no statistically significant difference between the mean performances of the two models.

The alternative hypothesis is

$$H_1 : \mu_A \neq \mu_B.$$

This is a two-tailed independent t-test.

7 Step 2: Calculate the Mean Accuracy

For Model A:

$$\bar{X}_A = \frac{80 + 82 + 78 + 85 + 80}{5}$$

$$\bar{X}_A = \frac{405}{5} = 81.$$

Therefore,

$$\boxed{\bar{X}_A = 81\%}$$

For Model B:

$$\bar{X}_B = \frac{70 + 72 + 68 + 75 + 70}{5}$$

$$\bar{X}_B = \frac{355}{5} = 71.$$

Therefore,

$$\boxed{\bar{X}_B = 71\%}$$

The observed difference between the two models is

$$\bar{X}_A - \bar{X}_B = 81 - 71 = 10.$$

Thus, Model A has a mean accuracy that is

$$\boxed{10 \text{ percentage points higher}}$$

than Model B.

However, the important question is:

Is this 10 percentage-point difference statistically significant, or could it have occurred due to random variation?

The independent t-test helps us answer this question.

8 Step 3: Calculate the Variance of Model A

The sample variance is

$$s_A^2 = \frac{\sum (X_i - \bar{X}_A)^2}{n_A - 1}.$$

For Model A:

X_i	$X_i - \bar{X}_A$	$(X_i - \bar{X}_A)^2$
80	-1	1
82	1	1
78	-3	9
85	4	16
80	-1	1
		28

Therefore,

$$s_A^2 = \frac{28}{5-1}$$

$$\boxed{s_A^2 = 7}.$$

9 Step 4: Calculate the Variance of Model B

For Model B, the mean accuracy is

$$\bar{X}_B = 71.$$

X_i	$X_i - \bar{X}_B$	$(X_i - \bar{X}_B)^2$
70	-1	1
72	1	1
68	-3	9
75	4	16
70	-1	1
		28

Therefore,

$$s_B^2 = \frac{28}{5-1}$$

$$\boxed{s_B^2 = 7}.$$

10 Step 5: Calculate the Pooled Variance

Assuming equal population variances, the pooled variance is

$$s_p^2 = \frac{(n_A - 1)s_A^2 + (n_B - 1)s_B^2}{n_A + n_B - 2}.$$

Substituting the values:

$$s_p^2 = \frac{(5-1)(7) + (5-1)(7)}{5+5-2}.$$

Therefore,

$$s_p^2 = \frac{28+28}{8}$$

$$\boxed{s_p^2 = 7}.$$

11 Step 6: Calculate the Standard Error

The standard error of the difference between the two means is

$$SE = \sqrt{s_p^2 \left(\frac{1}{n_A} + \frac{1}{n_B} \right)}.$$

Substituting:

$$SE = \sqrt{7 \left(\frac{1}{5} + \frac{1}{5} \right)}.$$

$$SE = \sqrt{7(0.4)}$$

$$SE = \sqrt{2.8}$$

$$\boxed{SE = 1.673}.$$

12 Step 7: Calculate the t-Statistic

The independent t-test statistic is

$$t = \frac{\bar{X}_A - \bar{X}_B}{SE}.$$

Substituting the values:

$$t = \frac{81 - 71}{1.673}.$$

Therefore,

$$\boxed{t = 5.976}.$$

13 Step 8: Calculate Degrees of Freedom

The degrees of freedom are

$$df = n_A + n_B - 2.$$

Therefore,

$$df = 5 + 5 - 2$$

$$\boxed{df = 8}.$$

14 Step 9: Statistical Decision

For a two-tailed test with

$$\alpha = 0.05$$

and

$$df = 8,$$

the critical t-value is approximately

$$t_{\text{critical}} = 2.306.$$

The calculated value is

$$t_{\text{calculated}} = 5.976.$$

Since

$$|t_{\text{calculated}}| > t_{\text{critical}},$$

we obtain

$$5.976 > 2.306.$$

Therefore, we reject the null hypothesis.

15 Final Research Conclusion

There is a **statistically significant difference** between the mean accuracies of Model A (SVM) and Model B (Logistic Regression) at the 5% significance level.

Since Model A has a higher mean accuracy:

$$81\% > 71\%,$$

the experiment provides statistical evidence that the performance of the two models differs.
Therefore,

Model A performs significantly differently from Model B.

16 What Does the t-Test Actually Tell Us?

The independent t-test does not simply compare the two means.

It considers both:

1. Difference between the two means.
2. Variability within the two groups.

The t-statistic can be understood as

$$t = \frac{\text{Difference between means}}{\text{Standard error of the difference}}$$

In this example:

$$\text{Difference between means} = 10$$

and

$$SE = 1.673.$$

Therefore,

$$t = \frac{10}{1.673} = 5.976.$$

A large t-value indicates that the observed difference between the two model performances is large relative to the variability in the measurements.

17 Important Machine Learning Example: When NOT to Use It

Suppose we evaluate two models on the same five datasets:

Dataset	Model A	Model B
D1	0.90	0.85
D2	0.92	0.87
D3	0.88	0.84
D4	0.91	0.89
D5	0.94	0.90

Here, each Model A result is directly paired with a Model B result from the same dataset. For example,

$$(D1 : 0.90, 0.85)$$

forms one pair.

Therefore, these observations are **not independent**.

An independent t-test should generally not be used for this situation.

Instead, a

Paired t-test

can be considered.

If the normality assumption for the paired differences is not reasonable, a

Wilcoxon signed-rank test

may be considered.

18 Key Rule for Machine Learning Students

Independent observations → Independent t-test

Paired observations → Paired t-test

For ML experiments:

Independent test samples/independent experiments → Independent t-test

Same test samples/datasets → Paired t-test

19 Summary

Parameter	Result
Model A mean accuracy	81%
Model B mean accuracy	71%
Difference	10 percentage points
Pooled variance	7
Standard error	1.673
Calculated t-value	5.976
Degrees of freedom	8
Critical t-value	2.306
Significance level	0.05
Decision	Reject H_0
Conclusion	Significant difference

20 Paired t-Test in Machine Learning

20.1 What is a Paired t-Test?

A paired t-test is used when two sets of observations are **related or paired**. In machine learning research, it is commonly used when two models are evaluated on the **same datasets, test samples, or experimental conditions**.

For example, if Model A and Model B are evaluated on the same benchmark datasets, each dataset produces one performance value for Model A and one performance value for Model B. These two values form a pair.

The paired t-test determines whether the **mean difference** between the two models is statistically significant.

20.2 When is a Paired t-Test Used?

A paired t-test can be used when:

- Two ML models are evaluated on the same datasets.
- Two classification algorithms are tested on the same test samples.
- Two regression models are evaluated on the same benchmark datasets.
- Two image-quality assessment models are evaluated on the same images.
- Two video-quality assessment models are evaluated on the same videos.

20.3 Simple Rule

Same datasets $\rightarrow$ Paired t-Test

Independent datasets/groups $\rightarrow$ Independent t-Test

20.4 Numerical Example

Suppose a researcher wants to compare two image-classification models:

- **Model A:** Proposed CNN
- **Model B:** Existing CNN

Both models are evaluated on the **same eight datasets**. The classification accuracy (%) obtained by the two models is shown below.

Dataset	Model A	Model B
D1	82	80
D2	85	83
D3	88	84
D4	90	87
D5	86	85
D6	91	88
D7	84	82
D8	89	86

Since both models are evaluated on the same datasets, the observations are paired. Therefore, a paired t-test is appropriate.

We use

$$\alpha = 0.05.$$

20.5 Step 1: Formulate the Hypotheses

Let

$$d_i = A_i - B_i$$

represent the difference between Model A and Model B for dataset i .

The null hypothesis is

$$H_0 : \mu_d = 0.$$

This means that there is no statistically significant difference between the average performances of the two models.

The alternative hypothesis is

$$H_1 : \mu_d \neq 0.$$

This means that there is a statistically significant difference between the average performances of the two models.

20.6 Step 2: Calculate the Differences

For each dataset:

$$d_i = A_i - B_i.$$

Dataset	Model A	Model B	Difference d_i
D1	82	80	2
D2	85	83	2
D3	88	84	4
D4	90	87	3
D5	86	85	1
D6	91	88	3
D7	84	82	2
D8	89	86	3

Thus,

$$d = \{2, 2, 4, 3, 1, 3, 2, 3\}.$$

There are

$$n = 8$$

paired observations.

20.7 Step 3: Calculate the Mean Difference

The mean difference is

$$\bar{d} = \frac{\sum d_i}{n}.$$

Therefore,

$$\begin{aligned}\bar{d} &= \frac{2 + 2 + 4 + 3 + 1 + 3 + 2 + 3}{8} \\ &= \frac{20}{8}\end{aligned}$$

$$\boxed{\bar{d} = 2.5}.$$

Thus, Model A has an average accuracy that is

$$\boxed{2.5 \text{ percentage points higher}}$$

than Model B.

20.8 Step 4: Calculate the Variance of the Differences

The sample variance of the differences is

$$s_d^2 = \frac{\sum (d_i - \bar{d})^2}{n - 1}.$$

Since

$$\bar{d} = 2.5,$$

we obtain:

d_i	$d_i - \bar{d}$	$(d_i - \bar{d})^2$
2	-0.5	0.25
2	-0.5	0.25
4	1.5	2.25
3	0.5	0.25
1	-1.5	2.25
3	0.5	0.25
2	-0.5	0.25
3	0.5	0.25
		6.00

Therefore,

$$s_d^2 = \frac{6}{8-1}$$

$$\boxed{s_d^2 = 0.8571}.$$

20.9 Step 5: Calculate the Standard Deviation

$$s_d = \sqrt{s_d^2}$$

$$s_d = \sqrt{0.8571}$$

$$\boxed{s_d \approx 0.926}.$$

20.10 Step 6: Calculate the Standard Error

The standard error of the mean difference is

$$SE_{\bar{d}} = \frac{s_d}{\sqrt{n}}.$$

Therefore,

$$SE_{\bar{d}} = \frac{0.926}{\sqrt{8}}$$

$$= \frac{0.926}{2.828}$$

$$\boxed{SE_{\bar{d}} \approx 0.3275}.$$

20.11 Step 7: Calculate the t-Statistic

The paired t-test statistic is

$$t = \frac{\bar{d} - \mu_{d,0}}{s_d/\sqrt{n}}.$$

Under the null hypothesis,

$$\mu_{d,0} = 0.$$

Therefore,

$$t = \frac{2.5 - 0}{0.3275}$$

$$\boxed{t \approx 7.63}.$$

20.12 Step 8: Calculate Degrees of Freedom

For a paired t-test,

$$df = n - 1.$$

Therefore,

$$df = 8 - 1$$

$$\boxed{df = 7}.$$

20.13 Step 9: Statistical Decision

For a two-tailed test with

$$\alpha = 0.05$$

and

$$df = 7,$$

the critical t-value is approximately

$$t_{\text{critical}} = 2.365.$$

The calculated value is

$$t_{\text{calculated}} = 7.63.$$

Since

$$|t_{\text{calculated}}| > t_{\text{critical}},$$

we have

$$7.63 > 2.365.$$

Therefore, we reject the null hypothesis:

$$\boxed{\text{Reject } H_0}.$$

20.14 Final Conclusion

There is a **statistically significant difference** between the performances of Model A and Model B at the 5% significance level.

The mean difference is

$$\bar{d} = 2.5.$$

Therefore, Model A achieves an average accuracy that is 2.5 percentage points higher than Model B.

Since

$$t(7) = 7.63,$$

the difference is statistically significant.

Hence,

$$\boxed{\text{Model A significantly outperforms Model B.}}$$

20.15 Why is this a Paired t-Test?

The important point is that both models are evaluated on the **same datasets**.

For example:

$$D1 : \quad 82 \quad \text{vs.} \quad 80$$

$$D2 : \quad 85 \quad \text{vs.} \quad 83$$

Each pair belongs to the same dataset.

Therefore, the observations are related.

We first calculate the difference:

$$d_i = A_i - B_i$$

and then perform a t-test on these differences.

Thus, the paired t-test can be viewed as a

one-sample t-test on the paired differences.

20.16 Independent vs. Paired t-Test in ML

Situation	Appropriate Test
Two independent groups	Independent t-test
Two models on the same datasets	Paired t-test
Before and after measurement	Paired t-test
Three or more independent groups	One-way ANOVA
Three or more models on same datasets	Friedman test

Rule to Remember

Same datasets $\rightarrow$ Paired t-Test

Independent datasets/groups $\rightarrow$ Independent t-Test

20.17 Wilcoxon Signed-Rank Test

The Wilcoxon signed-rank test is a non-parametric test used to compare two paired sets of observations. It is particularly useful when the paired differences do not satisfy the normality assumption required for the paired t-test.

Examples of Wilcoxon Signed-Rank Test with Accuracies

Case 1: Algorithm B is Better

Accuracies:

$$A = [0.82, 0.85, 0.80, 0.83, 0.81], \quad B = [0.88, 0.86, 0.84, 0.87, 0.85]$$

Step 1: Differences

$$d = B - A = [0.06, 0.01, 0.04, 0.04, 0.04]$$

Step 2: Absolute differences

$$|d| = [0.06, 0.01, 0.04, 0.04, 0.04]$$

Step 3: Ranks

$0.01 \rightarrow 1, \quad 0.06 \rightarrow 2, \quad 0.04 \rightarrow 3, 4, 5 \text{ (avg} = 4 \text{ each)}$

$$R = [2, 1, 4, 4, 4]$$

Step 4: Signed ranks All $d_i > 0 \Rightarrow$ all ranks positive.

Step 5: Rank sums

$$W^+ = 15, \quad W^- = 0, \quad W = \min(W^+, W^-) = 0$$

Decision: Since all ranks favor B, **Algorithm B is significantly better.**

—

Case 2: Algorithm A is Better

Accuracies:

$$A = [0.90, 0.88, 0.92, 0.91, 0.89], \quad B = [0.85, 0.84, 0.88, 0.86, 0.83]$$

Step 1: Differences

$$d = B - A = [-0.05, -0.04, -0.04, -0.05, -0.06]$$

Step 2: Absolute differences

$$|d| = [0.05, 0.04, 0.04, 0.05, 0.06]$$

Step 3: Ranks

$$0.04 \rightarrow 1.5, 1.5, \quad 0.05 \rightarrow 3.5, 3.5, \quad 0.06 \rightarrow 5$$

$$R = [3.5, 1.5, 1.5, 3.5, 5]$$

Step 4: Signed ranks All $d_i < 0 \Rightarrow$ all ranks negative.

Step 5: Rank sums

$$W^+ = 0, \quad W^- = 15, \quad W = \min(W^+, W^-) = 0$$

Decision: Since all ranks favor A, **Algorithm A is significantly better.**

21 Comparing Two Classification Algorithms

Classifier evaluation involves two key tasks:

1. **Assessing a single classification algorithm:** How well does it perform on its own?
2. **Comparing two classification algorithms:** Is one truly better than the other, or is the observed difference due to chance?

We first describe how to assess one classifier using standard metrics derived from the confusion matrix. We then present three common statistical tests used to compare classifiers

Measuring Classifier Performance

1. Assessing a Single Classifier

The performance of one classifier is typically evaluated using a **confusion matrix**:

	Predicted Positive	Predicted Negative
Actual Positive	TP	FN
Actual Negative	FP	TN

From this we define:

- **Accuracy:** $\frac{TP + TN}{TP + TN + FP + FN}$
- **Precision:** $\frac{TP}{TP + FP}$
- **Recall (Sensitivity):** $\frac{TP}{TP + FN}$
- **Specificity:** $\frac{TN}{TN + FP}$
- **F1-score:** Harmonic mean of Precision and Recall
- ROC curves, AUC, Cohen's κ , etc.

These metrics tell us how good a classifier is in isolation.

2. Comparing Two Classification Algorithms

When two classifiers are trained and tested on the same or similar datasets, simple accuracy comparisons are insufficient. We need **statistical tests** to determine if observed differences are significant.

We compare two algorithms (say Algorithm A and Algorithm B) to check if the observed performance differences are **statistically significant**. Below, common tests are explained with detailed examples.

One-Way ANOVA Test

Algorithm

1. Collect k groups (algorithms) with n observations each. Denote them as X_{ij} where $i = 1, 2, \dots, k$ and $j = 1, 2, \dots, n$.
2. Compute group means:

$$\bar{X}_i = \frac{1}{n} \sum_{j=1}^n X_{ij}, \quad i = 1, 2, \dots, k$$

3. Compute grand mean:

$$\bar{X} = \frac{1}{kn} \sum_{i=1}^k \sum_{j=1}^n X_{ij}$$

4. Compute **Between-group Sum of Squares**:

$$SS_B = n \sum_{i=1}^k (\bar{X}_i - \bar{X})^2$$

5. Compute **Within-group Sum of Squares**:

$$SS_W = \sum_{i=1}^k \sum_{j=1}^n (X_{ij} - \bar{X}_i)^2$$

6. Degrees of freedom:

$$df_B = k - 1, \quad df_W = N - k \quad (N = kn)$$

7. Compute Mean Squares:

$$MS_B = \frac{SS_B}{df_B}, \quad MS_W = \frac{SS_W}{df_W}$$

8. Compute F -statistic:

$$F = \frac{MS_B}{MS_W}$$

9. Decision rule: Compare F with $F_{critical}$ at chosen significance level.

- If $F > F_{critical}$, reject H_0 : at least one mean is different.
- Otherwise, fail to reject H_0 : no significant difference.

Examples of One-Way ANOVA Outcomes

Case 1: Significant Difference (One-Way ANOVA)

We compare the means of three algorithms A , B , and C .

$$A = [82, 85, 88, 86], \quad B = [79, 81, 80, 78], \quad C = [84, 83, 85, 82]$$

Step 1: Compute Group Means

$$\bar{A} = \frac{82 + 85 + 88 + 86}{4} = \frac{341}{4} = 85.25$$

$$\bar{B} = \frac{79 + 81 + 80 + 78}{4} = \frac{318}{4} = 79.5$$

$$\bar{C} = \frac{84 + 83 + 85 + 82}{4} = \frac{334}{4} = 83.5$$

Overall mean:

$$\bar{X} = \frac{341 + 318 + 334}{12} = \frac{993}{12} = 82.75$$

Step 2: Between-Group Sum of Squares (SS_B)

$$SS_B = n[(\bar{A} - \bar{X})^2 + (\bar{B} - \bar{X})^2 + (\bar{C} - \bar{X})^2]$$

where $n = 4$ (number of samples per group).

$$(\bar{A} - \bar{X})^2 = (85.25 - 82.75)^2 = (2.5)^2 = 6.25$$

$$(\bar{B} - \bar{X})^2 = (79.5 - 82.75)^2 = (-3.25)^2 = 10.5625$$

$$(\bar{C} - \bar{X})^2 = (83.5 - 82.75)^2 = (0.75)^2 = 0.5625$$

$$SS_B = 4(6.25 + 10.5625 + 0.5625) = 4(17.375) = 69.5$$

Step 3: Within-Group Sum of Squares (SS_W)

For each group, compute deviations from its mean:

- For A:

$$\begin{aligned} & (82 - 85.25)^2 + (85 - 85.25)^2 + (88 - 85.25)^2 + (86 - 85.25)^2 \\ &= (-3.25)^2 + (-0.25)^2 + (2.75)^2 + (0.75)^2 \\ &= 10.5625 + 0.0625 + 7.5625 + 0.5625 = 18.75 \end{aligned}$$

- For B:

$$\begin{aligned} & (79 - 79.5)^2 + (81 - 79.5)^2 + (80 - 79.5)^2 + (78 - 79.5)^2 \\ &= (-0.5)^2 + (1.5)^2 + (0.5)^2 + (-1.5)^2 \\ &= 0.25 + 2.25 + 0.25 + 2.25 = 5.0 \end{aligned}$$

- For C:

$$\begin{aligned} & (84 - 83.5)^2 + (83 - 83.5)^2 + (85 - 83.5)^2 + (82 - 83.5)^2 \\ &= (0.5)^2 + (-0.5)^2 + (1.5)^2 + (-1.5)^2 \\ &= 0.25 + 0.25 + 2.25 + 2.25 = 5.0 \end{aligned}$$

Total within-group:

$$SS_W = 18.75 + 5.0 + 5.0 = 28.75$$

Step 4: Degrees of Freedom

$$df_B = k - 1 = 3 - 1 = 2, \quad df_W = N - k = 12 - 3 = 9$$

Step 5: Mean Squares

$$\begin{aligned} MS_B &= \frac{SS_B}{df_B} = \frac{69.5}{2} = 34.75 \\ MS_W &= \frac{SS_W}{df_W} = \frac{28.75}{9} \approx 3.19 \end{aligned}$$

Step 6: F-statistic

$$F = \frac{MS_B}{MS_W} = \frac{34.75}{3.19} \approx 10.90$$

Step 7: Decision From F-tables:

$$F_{0.05,2,9} = 4.26$$

Since

$$F = 10.90 > 4.26,$$

we reject H_0 .

Conclusion: At least one algorithm differs significantly in mean performance from the others.

Case 2: No Significant Difference

$$A = [82, 83, 81, 82], \quad B = [81, 82, 80, 82], \quad C = [82, 82, 81, 83]$$

Step 1: Means

$$\bar{A} = 82, \quad \bar{B} = 81.25, \quad \bar{C} = 82, \quad \bar{X} = 81.75$$

Step 2: Between-group variability

$$SS_B = 1.5$$

Step 3: Within-group variability

$$SS_W = 7.5$$

Step 4: Degrees of freedom

$$df_B = 2, \quad df_W = 9$$

Step 5: Mean Squares

$$MS_B = 0.75, \quad MS_W = 0.83$$

Step 6: F-statistic

$$F = \frac{0.75}{0.83} = 0.90$$

Step 7: Decision $F = 0.90 < 4.26$. **No significant difference between algorithms.**

Case 3: Borderline Case

$$A = [84, 85, 83, 84], \quad B = [81, 82, 80, 81], \quad C = [83, 84, 82, 83]$$

Step 1: Means

$$\bar{A} = 84, \quad \bar{B} = 81, \quad \bar{C} = 83, \quad \bar{X} = 82.67$$

Step 2: Between-group variability

$$SS_B = 18.67$$

Step 3: Within-group variability

$$SS_W = 6$$

Step 4: Degrees of freedom

$$df_B = 2, \quad df_W = 9$$

Step 5: Mean Squares

$$MS_B = 9.33, \quad MS_W = 0.67$$

Step 6: F-statistic

$$F = \frac{9.33}{0.67} = 13.93$$

Step 7: Decision $F = 13.93 > 4.26$. **Significant difference detected (closer to borderline).**

Summary of Cases

Case	Group Means	SS_B	SS_W	Decision
1: Significant	(85.25, 79.5, 83.5)	69.48	20.25	Significant difference
2: No Diff.	(82, 81.25, 82)	1.5	7.5	No significant difference
3: Borderline	(84, 81, 83)	18.67	6	Significant (borderline)

Friedman Test

What is the Friedman Test?

It is used to compare the performance of k algorithms across N datasets (or subjects) without assuming normality.

Algorithm

1. Arrange results of k algorithms across N datasets in a table.
2. For each dataset, assign ranks ($1 = \text{best}$, $k = \text{worst}$; ties get average ranks).
3. Compute the average rank of each algorithm:

$$R_j = \frac{1}{N} \sum_{i=1}^N r_{ij}, \quad j = 1, \dots, k$$

4. Compute Friedman statistic:

$$\chi_F^2 = \frac{12}{Nk(k+1)} \left(\sum_{j=1}^k R_j^2 \right) - 3N(k+1)$$

5. Compare with chi-square critical value $\chi_{k-1, \alpha}^2$. If $\chi_F^2 > \chi_{k-1, \alpha}^2$, reject H_0 .

Examples of Friedman Test Outcomes

Case 1: Significant Difference (Friedman Test)

Data (Accuracy)

$$A = [82, 85, 88, 86], \quad B = [79, 81, 80, 78], \quad C = [84, 83, 85, 82]$$

We have $k = 3$ algorithms (treatments) tested on $N = 4$ datasets (blocks). **Step 1: Ranking within each dataset**

For each dataset, assign ranks (1 = best accuracy, 3 = worst accuracy).

Dataset	A	B	C
D1	82 $\rightarrow$ 2	79 $\rightarrow$ 3	84 $\rightarrow$ 1
D2	85 $\rightarrow$ 1	81 $\rightarrow$ 3	83 $\rightarrow$ 2
D3	88 $\rightarrow$ 1	80 $\rightarrow$ 3	85 $\rightarrow$ 2
D4	86 $\rightarrow$ 1	78 $\rightarrow$ 3	82 $\rightarrow$ 2

Step 2: Sum and Average of Ranks

$$R_A = (2 + 1 + 1 + 1) = 5, \quad \bar{R}_A = \frac{5}{4} = 1.25$$

$$R_B = (3 + 3 + 3 + 3) = 12, \quad \bar{R}_B = \frac{12}{4} = 3.0$$

$$R_C = (1 + 2 + 2 + 2) = 7, \quad \bar{R}_C = \frac{7}{4} = 1.75$$

So, the average ranks are:

$$\bar{R}_A = 1.25, \quad \bar{R}_B = 3.0, \quad \bar{R}_C = 1.75$$

Step 3: Friedman Test Statistic

The Friedman statistic is

$$\chi_F^2 = \frac{12}{Nk(k+1)} \sum_{j=1}^k R_j^2 - 3N(k+1)$$

Substitute values: $N = 4$, $k = 3$, $R_A = 5$, $R_B = 12$, $R_C = 7$.

$$\sum R_j^2 = 5^2 + 12^2 + 7^2 = 25 + 144 + 49 = 218$$

$$\chi_F^2 = \frac{12}{(4)(3)(4)}(218) - 3(4)(4)$$

$$\chi_F^2 \approx 6.48$$

Step 4: Decision Rule

Degrees of freedom:

$$df = k - 1 = 3 - 1 = 2$$

Critical value at $\alpha = 0.05$:

$$\chi_{0.05,2}^2 = 5.99$$

Since

$$\chi_F^2 = 6.48 > 5.99,$$

we reject H_0 . **Conclusion:** There is a **significant difference** among the algorithms. The algorithm with the **lowest average rank** is best:

Best = Algorithm A (rank 1.25).

Case 2: No Significant Difference (Friedman Test)

Data (Accuracy)

$$A = [82, 83, 81, 82], \quad B = [81, 82, 80, 82], \quad C = [82, 82, 81, 83]$$

We have $k = 3$ algorithms tested on $N = 4$ datasets.

Step 1: Ranking within each dataset

Highest accuracy = rank 1, lowest = rank 3, ties receive the average of the ranks they would have occupied.

Dataset	A	B	C
D1	1.5	3	1.5
D2	1	2.5	2.5
D3	1.5	3	1.5
D4	2.5	2.5	1

Step 2: Rank sums

$$R_A = 1.5 + 1 + 1.5 + 2.5 = 6.5$$

$$R_B = 3 + 2.5 + 3 + 2.5 = 11$$

$$R_C = 1.5 + 2.5 + 1.5 + 1 = 6.5$$

Step 3: Friedman statistic

The Friedman test statistic is

$$\chi_F^2 = \frac{12}{Nk(k+1)} \left(\sum_{j=1}^k R_j^2 \right) - 3N(k+1)$$

First compute squared rank sums:

$$R_A^2 = (6.5)^2 = 42.25, \quad R_B^2 = (11)^2 = 121, \quad R_C^2 = (6.5)^2 = 42.25$$

$$\sum_{j=1}^k R_j^2 = 42.25 + 121 + 42.25 = 205.5$$

Now substitute values ($N = 4$, $k = 3$):

$$\begin{aligned} \chi_F^2 &= \frac{12}{4 \cdot 3 \cdot 4} (205.5) - 3(4)(4) \\ &= \frac{12}{48} (205.5) - 48 = 0.25 \times 205.5 - 48 = 51.375 - 48 = 3.375 \end{aligned}$$

Step 4: Decision

Degrees of freedom:

$$df = k - 1 = 2$$

Critical value at $\alpha = 0.05$:

$$\chi_{0.05,2}^2 = 5.99$$

Since

$$\chi_F^2 = 3.375 < 5.99,$$

we fail to reject H_0 . **Conclusion:** There is **no significant difference** among the three algorithms. All perform similarly across the datasets.

Case 3: Borderline Case (Friedman Test)

Data (Accuracy)

$$A = [84, 85, 83, 84], \quad B = [81, 82, 80, 81], \quad C = [83, 84, 82, 83]$$

We have $k = 3$ algorithms tested on $N = 4$ datasets.

Step 1: Ranking within each dataset

Highest accuracy = rank 1, lowest = rank 3. Since the values are well separated, there are no ties.

Dataset	A	B	C
D1	1	3	2
D2	1	3	2
D3	1	3	2
D4	1	3	2

Step 2: Rank sums and average ranks

$$R_A = 1 + 1 + 1 + 1 = 4, \quad R_B = 3 + 3 + 3 + 3 = 12, \quad R_C = 2 + 2 + 2 + 2 = 8$$

Average ranks:

$$\bar{R}_A = \frac{4}{4} = 1.0, \quad \bar{R}_B = \frac{12}{4} = 3.0, \quad \bar{R}_C = \frac{8}{4} = 2.0$$

Step 3: Friedman statistic

The Friedman test statistic is:

$$\chi_F^2 = \frac{12}{Nk(k+1)} \left(\sum_{j=1}^k R_j^2 \right) - 3N(k+1)$$

Compute squared rank sums:

$$R_A^2 = 4^2 = 16, \quad R_B^2 = 12^2 = 144, \quad R_C^2 = 8^2 = 64$$

$$\sum_{j=1}^k R_j^2 = 16 + 144 + 64 = 224$$

Now substitute ($N = 4$, $k = 3$):

$$\begin{aligned} \chi_F^2 &= \frac{12}{4 \cdot 3 \cdot 4} (224) - 3(4)(4) \\ &= \frac{12}{48} (224) - 48 = 0.25 \times 224 - 48 = 56 - 48 = 8.0 \end{aligned}$$

Step 4: Decision

Degrees of freedom:

$$df = k - 1 = 2$$

Critical value at $\alpha = 0.05$:

$$\chi_{0.05,2}^2 = 5.99$$

Since

$$\chi_F^2 = 8.0 > 5.99,$$

we reject H_0 . **Conclusion:** There is a **significant difference** among the three algorithms. From the average ranks:

$$A (1.0) < C (2.0) < B (3.0)$$

Thus, **Algorithm A** performs best, **B** worst, and **C** is moderate.

Summary of Cases

Case	Average Ranks ($\bar{R}_A, \bar{R}_B, \bar{R}_C$)	χ_F^2	Decision
1: Significant	(1.25, 3.00, 1.75)	6.48	Significant; A best, B worst
2: No Difference	(1.625, 2.75, 1.625)	3.38	No significant difference
3: Borderline	(1.00, 3.00, 2.00)	8.00	Significant; A best, B worst, C moderate

Contingency Table / Correctness Matrix and McNemar's Test

1. Introduction

When two classification models are evaluated on the **same test set**, their predictions are paired because both models make predictions for exactly the same test samples.

Suppose we have two classifiers:

- Classifier A: Proposed Model
- Classifier B: Baseline Model

A simple comparison of their accuracies may not be sufficient to determine whether the difference in performance is statistically significant.

For example, suppose:

$$\text{Accuracy}_A = 85\%$$

and

$$\text{Accuracy}_B = 70\%.$$

Although Classifier A appears to be better by 15 percentage points, we need to determine whether this observed improvement is statistically significant.

For paired predictions on the same test set, one commonly used statistical test is the **McNemar's test**.

2. What Does McNemar's Test Do?

McNemar's test is a non-parametric statistical test used to compare the performance of two classifiers evaluated on the **same test samples**.

Unlike a conventional comparison of two accuracy values, McNemar's test focuses only on the samples for which the two classifiers produce **different correctness outcomes**.

In other words, it asks:

When the two classifiers disagree in terms of correctness, does one classifier win significantly more often than the other?

There are four possible situations for every test sample:

1. Both classifiers are correct.
2. Classifier A is correct and Classifier B is wrong.
3. Classifier A is wrong and Classifier B is correct.
4. Both classifiers are wrong.

McNemar's test is primarily concerned with cases 2 and 3.

These are called the **discordant pairs**.

3. Numerical Example

Consider two binary classifiers, A and B, evaluated on a test set containing 20 samples.
The actual class labels and predictions are shown below.

Table 2: Actual labels and predictions of two classifiers

Sample	Actual	A Prediction	B Prediction	A Correct?	B Correct?
1	1	1	1	Yes	Yes
2	1	1	1	Yes	Yes
3	0	0	0	Yes	Yes
4	0	0	0	Yes	Yes
5	1	1	0	Yes	No
6	0	0	1	Yes	No
7	1	1	0	Yes	No
8	0	0	1	Yes	No
9	1	1	1	Yes	Yes
10	0	1	0	No	Yes
11	1	1	1	Yes	Yes
12	0	0	0	Yes	Yes
13	1	0	1	No	Yes
14	0	1	0	No	Yes
15	1	1	1	Yes	Yes
16	0	0	0	Yes	Yes
17	1	1	0	Yes	No
18	0	0	1	Yes	No
19	1	1	1	Yes	Yes
20	0	0	0	Yes	Yes

4. Accuracy of Classifier A

Classifier A makes 17 correct predictions out of 20 samples.
Therefore,

$$\text{Accuracy}_A = \frac{17}{20} = 0.85$$

or

$\text{Accuracy}_A = 85\%$

5. Accuracy of Classifier B

Classifier B makes 14 correct predictions out of 20 samples.
Therefore,

$$\text{Accuracy}_B = \frac{14}{20} = 0.70$$

or

$\text{Accuracy}_B = 70\%$

The observed difference in accuracy is therefore

$$85\% - 70\% = 15\%.$$

Thus,

$$\Delta Accuracy = 15\%$$

However, this difference alone does not tell us whether the improvement is statistically significant.

6. Constructing the Correctness Contingency Table

For each test sample, we record whether Classifier A and Classifier B are correct or incorrect.

The resulting 2×2 contingency table is:

	B Correct	B Wrong
A Correct	n_{11}	n_{10}
A Wrong	n_{01}	n_{00}

For our example:

- n_{11} = both A and B are correct.
- n_{10} = A is correct but B is wrong.
- n_{01} = A is wrong but B is correct.
- n_{00} = both A and B are wrong.

From the prediction table:

$$n_{11} = 11$$

$$n_{10} = 6$$

$$n_{01} = 3$$

$$n_{00} = 0$$

Therefore, the correctness contingency table becomes:

	B Correct	B Wrong
A Correct	11	6
A Wrong	3	0

We can verify the total number of samples:

$$11 + 6 + 3 + 0 = 20.$$

7. Understanding the Discordant Pairs

The most important aspect of McNemar's test is that it focuses on the **discordant pairs**.

These are:

$$n_{10} = 6$$

and

$$n_{01} = 3.$$

The interpretation is:

$$n_{10} = 6 :$$

Classifier A is correct while Classifier B is wrong for 6 samples.
Therefore, A “wins” over B on 6 discordant samples.
Similarly,

$$n_{01} = 3 :$$

Classifier B is correct while Classifier A is wrong for 3 samples.
Therefore, B “wins” over A on 3 discordant samples.
Hence,

$$\boxed{\text{A wins} = 6}$$

and

$$\boxed{\text{B wins} = 3}.$$

The total number of discordant observations is

$$n_d = n_{10} + n_{01}.$$

Therefore,

$$n_d = 6 + 3 = 9.$$

Thus, only 9 out of the 20 samples contribute to the McNemar test.

8. Null and Alternative Hypotheses

The null hypothesis is

$$H_0 : n_{10} = n_{01}.$$

In words:

There is no significant difference between the two classifiers in terms of their discordant predictions.

The alternative hypothesis is

$$H_1 : n_{10} \neq n_{01}.$$

In words:

There is a statistically significant difference between the two classifiers.

For a two-sided test, we normally use

$$\alpha = 0.05.$$

The test statistic is

$$\chi^2 = \frac{(n_{10} - n_{01})^2}{n_{10} + n_{01}}.$$

For our example,

$$n_{10} = 6$$

and

$$n_{01} = 3.$$

Therefore,

$$\begin{aligned}\chi^2 &= \frac{(6-3)^2}{6+3} \\ &= \frac{3^2}{9} \\ &= \frac{9}{9}.\end{aligned}$$

Hence,

$$\boxed{\chi^2 = 1.00}.$$

The degrees of freedom are

$$df = 1.$$

At significance level

$$\alpha = 0.05,$$

the critical chi-square value is approximately

$$\chi_{0.05,1}^2 = 3.841.$$

Since

$$1.00 < 3.841,$$

we fail to reject the null hypothesis.

Therefore, the difference is not statistically significant.

LIME: Mathematical Intuition and Detailed Numerical Example

1 LIME: A Detailed Numerical Example

1.1 Introduction to LIME

LIME stands for **Local Interpretable Model-agnostic Explanations**. It is an explainable artificial intelligence (XAI) technique used to explain the prediction made by a complex machine learning model.

Suppose that a machine learning model has been trained to predict whether a student will pass an examination. The model may be a Random Forest, Support Vector Machine, XGBoost model, neural network, or any other complex model. The model can make accurate predictions, but it may be difficult to understand why a particular prediction was made.

LIME addresses this problem by constructing a simple and interpretable model around the particular instance that we want to explain.

The main idea can be summarized as:

Complex Model $\rightarrow$ Generate Nearby Samples $\rightarrow$ Obtain Predictions
 $\rightarrow$ Assign Weights $\rightarrow$ Fit Simple Model $\rightarrow$ Explain Prediction

(1)

The important point is that LIME explains the model **locally**, rather than attempting to explain the entire model globally.

1.2 Problem Statement

Consider a trained machine learning model that predicts the probability that a student will pass an examination.

We consider three input features:

- x_1 : Number of study hours per day
- x_2 : Attendance percentage
- x_3 : Previous examination score

For a particular student, the input is

$$x = [8 \quad 85 \quad 70]. \quad (2)$$

Thus,

$$x_1 = 8, \quad (3)$$

$$x_2 = 85, \quad (4)$$

$$x_3 = 70. \quad (5)$$

Suppose that the trained black-box machine learning model gives the following prediction:

$$f(x) = 0.90. \quad (6)$$

Therefore, the model predicts that this student has a

$$\boxed{90\%} \tag{7}$$

probability of passing the examination.

The student now asks:

Why did the machine learning model predict a 90% probability of passing?

LIME can be used to answer this question.

1.3 Step 1: Select the Instance to be Explained

The first step in LIME is to select the particular instance for which we want an explanation.

The selected student is

$$x = (8, 85, 70). \tag{8}$$

The black-box model gives

$$f(x) = 0.90. \tag{9}$$

It is important to understand that LIME is not trying to explain all possible students. It is trying to explain the prediction for this particular student.

Therefore,

$$\boxed{\text{LIME explanation is local to } x = (8, 85, 70).} \tag{10}$$

1.4 Step 2: Generate Perturbed Samples

The next step is to generate several samples that are similar to the original student.

The purpose of generating these samples is to observe how the black-box model behaves around the selected student.

For simplicity, suppose that we generate the following five perturbed samples:

Table 1: Original and perturbed samples			
Sample	Study Hours	Attendance	Previous Score
x	8	85	70
z_1	8	85	60
z_2	8	75	70
z_3	6	85	70
z_4	6	75	60
z_5	8	75	60

The original student is

$$x = (8, 85, 70). \tag{11}$$

The first perturbed sample is

$$z_1 = (8, 85, 60). \tag{12}$$

Here, only the previous examination score has been changed.

Similarly,

$$z_2 = (8, 75, 70), \tag{13}$$

where attendance has been changed.
The third perturbed sample is

$$z_3 = (6, 85, 70), \quad (14)$$

where study hours have been changed.
The remaining samples contain changes in two features.

1.5 Step 3: Obtain Predictions from the Black-Box Model

Each perturbed sample is passed through the original machine learning model.
The original black-box model is treated as a function:

$$f(\cdot). \quad (15)$$

We do not need to know how the model internally makes its decision. We only need its predictions.
Suppose the predictions are as follows:

Sample	Study Hours	Attendance	Previous Score	$f(z)$
x	8	85	70	0.90
z_1	8	85	60	0.82
z_2	8	75	70	0.78
z_3	6	85	70	0.84
z_4	6	75	60	0.65
z_5	8	75	60	0.72

For example,

$$f(z_1) = 0.82. \quad (16)$$

This means that when the previous examination score is reduced from 70 to 60, the black-box model's predicted probability decreases from

$$0.90 \rightarrow 0.82. \quad (17)$$

Similarly,

$$f(z_2) = 0.78, \quad (18)$$

when attendance is reduced from 85% to 75%.

Also,

$$f(z_3) = 0.84, \quad (19)$$

when study hours are reduced from 8 to 6 hours.

These changes provide information about how the model behaves around the selected student.

1.6 Step 4: Calculate the Distance from the Original Instance

LIME gives greater importance to samples that are close to the original instance.

The distance between the original instance x and a perturbed sample z can be calculated using a distance measure.

For illustration, assume that the features have already been normalized. We use the Euclidean distance:

$$d(x, z) = \sqrt{\sum_{j=1}^3 (x_j - z_j)^2}. \quad (20)$$

For example, consider the perturbed sample z_1 .
 Suppose that after normalization, the distance between the original instance x and z_1 is

$$d(x, z_1) = 0.50. \quad (21)$$

Similarly, suppose the distances for the remaining perturbed samples are as follows:

Table 3: Distances from the original instance

Sample	Distance $d(x, z)$
z_1	0.50
z_2	0.70
z_3	0.60
z_4	1.20
z_5	1.00

The smaller the distance, the more similar the perturbed sample is to the original instance.
 For example,

$$d(x, z_1) = 0.50 \quad (22)$$

means that z_1 is relatively close to the original instance.
 On the other hand,

$$d(x, z_4) = 1.20 \quad (23)$$

means that z_4 is farther away from the original instance.
 Therefore,

$$d(x, z_1) < d(x, z_4), \quad (24)$$

which means that z_1 is more similar to the original instance than z_4 .

1.7 Step 5: Calculate Proximity Weights

LIME converts the distance into a similarity or proximity weight.

A commonly used exponential kernel can be written as

$$\pi_x(z) = \exp\left(-\frac{d(x, z)^2}{\sigma^2}\right), \quad (25)$$

where σ is a kernel-width parameter.
 For simplicity, assume

$$\sigma = 1. \quad (26)$$

Therefore,

$$\pi_x(z) = e^{-d(x, z)^2}. \quad (27)$$

1.7.1 Weight for z_1

For z_1 ,

$$d(x, z_1) = 0.50. \quad (28)$$

Therefore,

$$\pi_x(z_1) = e^{-(0.50)^2} \quad (29)$$

$$= e^{-0.25} \quad (30)$$

$$\approx 0.7788. \quad (31)$$

Hence,

$$\boxed{\pi_x(z_1) \approx 0.779} \quad (32)$$

1.7.2 Weight for z_2

For z_2 ,

$$d(x, z_2) = 0.70. \quad (33)$$

Therefore,

$$\pi_x(z_2) = e^{-(0.70)^2} \quad (34)$$

$$= e^{-0.49} \quad (35)$$

$$\approx 0.6126. \quad (36)$$

Hence,

$$\boxed{\pi_x(z_2) \approx 0.613} \quad (37)$$

1.7.3 Weight for z_3

For z_3 ,

$$d(x, z_3) = 0.60. \quad (38)$$

Therefore,

$$\pi_x(z_3) = e^{-(0.60)^2} \quad (39)$$

$$= e^{-0.36} \quad (40)$$

$$\approx 0.6977. \quad (41)$$

Hence,

$$\boxed{\pi_x(z_3) \approx 0.698} \quad (42)$$

1.7.4 Weight for z_4

For z_4 ,

$$d(x, z_4) = 1.20. \quad (43)$$

Therefore,

$$\pi_x(z_4) = e^{-(1.20)^2} \quad (44)$$

$$= e^{-1.44} \quad (45)$$

$$\approx 0.2369. \quad (46)$$

Hence,

$$\pi_x(z_4) \approx 0.237 \quad (47)$$

1.7.5 Weight for z_5

For z_5 ,

$$d(x, z_5) = 1.00. \quad (48)$$

Therefore,

$$\pi_x(z_5) = e^{-(1.00)^2} \quad (49)$$

$$= e^{-1} \quad (50)$$

$$\approx 0.3679. \quad (51)$$

Hence,

$$\pi_x(z_5) \approx 0.368 \quad (52)$$

The complete table is:

Table 4: Predictions, distances, and proximity weights

Sample	$f(z)$	$d(x, z)$	$\pi_x(z)$
z_1	0.82	0.50	0.779
z_2	0.78	0.70	0.613
z_3	0.84	0.60	0.698
z_4	0.65	1.20	0.237
z_5	0.72	1.00	0.368

This table demonstrates an important principle of LIME:

$$\text{Closer sample} \Rightarrow \text{Higher weight} \quad (53)$$

and

$$\text{Farther sample} \Rightarrow \text{Lower weight.} \quad (54)$$

1.8 Step 6: Construct an Interpretable Model

The original black-box model may be very complicated.

For example, it could be a deep neural network:

$$f(x) = \text{Deep Neural Network}(x). \quad (55)$$

Instead of trying to understand this complicated function, LIME constructs a simple surrogate model.

For numerical features, we can use a linear model:

$$g(z) = \beta_0 + \beta_1 z_1 + \beta_2 z_2 + \beta_3 z_3. \quad (56)$$

Here:

- β_0 is the intercept,
- β_1 represents the local contribution of study hours,

- β_2 represents the local contribution of attendance,
- β_3 represents the local contribution of previous score.

The purpose of this model is not to reproduce the entire black-box model. Instead, its purpose is to approximate the black-box model **near the selected instance**.

1.9 Step 7: Weighted Linear Regression

LIME finds the coefficients of the simple model by giving more importance to samples that are close to the original instance.

The objective function can be written as

$$\mathcal{L}(f, g, \pi_x) = \sum_{i=1}^n \pi_x(z_i) [f(z_i) - g(z_i)]^2. \quad (57)$$

Here:

- $f(z_i)$ is the prediction of the original black-box model,
- $g(z_i)$ is the prediction of the simple interpretable model,
- $\pi_x(z_i)$ is the proximity weight,
- n is the number of perturbed samples.

LIME attempts to minimize

$$\boxed{\mathcal{L}(f, g, \pi_x) = \sum_{i=1}^n \pi_x(z_i) [f(z_i) - g(z_i)]^2} \quad (58)$$

so that the simple model behaves similarly to the black-box model in the neighborhood of x .

1.10 Step 8: Example of the Local Surrogate Model

Suppose that after solving the weighted regression problem, LIME obtains the following local model:

$$\boxed{g(z) = -0.20 + 0.040z_1 + 0.005z_2 + 0.003z_3} \quad (59)$$

Recall that

$$z_1 = \text{Study Hours}, \quad (60)$$

$$z_2 = \text{Attendance}, \quad (61)$$

$$z_3 = \text{Previous Score}. \quad (62)$$

Therefore,

$$g(z) = -0.20 + 0.040(\text{Study Hours}) + 0.005(\text{Attendance}) + 0.003(\text{Previous Score}). \quad (63)$$

This equation is much easier for a human to understand than the original black-box model.

1.11 Step 9: Calculate the Prediction for the Original Student

For the original student,

$$x = (8, 85, 70). \quad (64)$$

Substitute these values into the local surrogate model:

$$g(x) = -0.20 + 0.040(8) + 0.005(85) + 0.003(70) \quad (65)$$

$$= -0.20 + 0.320 + 0.425 + 0.210 \quad (66)$$

$$= 0.755. \quad (67)$$

Thus,

$$\boxed{g(x) = 0.755} \quad (68)$$

The original black-box model predicted

$$f(x) = 0.90, \quad (69)$$

while the local surrogate predicted

$$g(x) = 0.755. \quad (70)$$

The difference occurs because the surrogate is only an approximation of the black-box model. In a real LIME implementation, the quality of the local approximation is evaluated by measuring how well g reproduces f in the local neighborhood.

The purpose of g is primarily to provide an interpretable explanation.

1.12 Step 10: Interpret the Coefficients

The local model is

$$g(z) = -0.20 + 0.040z_1 + 0.005z_2 + 0.003z_3. \quad (71)$$

The coefficient of study hours is

$$\beta_1 = 0.040. \quad (72)$$

The coefficient is positive. Therefore, locally, increasing study hours tends to increase the model's prediction.

The coefficient of attendance is

$$\beta_2 = 0.005. \quad (73)$$

This is also positive. Therefore, locally, higher attendance tends to increase the prediction.

The coefficient of previous score is

$$\beta_3 = 0.003. \quad (74)$$

Again, this is positive.

Therefore, all three features have a positive local influence in this example.

1.13 Step 11: Compare Feature Importance

The absolute values of the coefficients are

$$|\beta_1| = 0.040, \quad (75)$$

$$|\beta_2| = 0.005, \quad (76)$$

$$|\beta_3| = 0.003. \quad (77)$$

Therefore,

$$0.040 > 0.005 > 0.003. \quad (78)$$

Hence, according to this illustrative local surrogate model,

Study Hours > Attendance > Previous Score

(79)

in terms of the magnitude of their local coefficients.

This means that study hours have the strongest local influence among the three features in this particular explanation.

1.14 Step 12: Positive and Negative Contributions

A major advantage of LIME is that it can identify both positive and negative contributions.

In general, a local linear model can be written as

$$g(x) = \beta_0 + \sum_{j=1}^p \beta_j x_j. \quad (80)$$

If

$$\beta_j > 0, \quad (81)$$

then feature j contributes positively to the local prediction.

If

$$\beta_j < 0, \quad (82)$$

then feature j contributes negatively to the local prediction.

For example, suppose LIME instead produced

$$g(x) = -0.10 + 0.040x_1 + 0.005x_2 - 0.020x_3. \quad (83)$$

Then:

$$\beta_1 = +0.040, \quad (84)$$

$$\beta_2 = +0.005, \quad (85)$$

$$\beta_3 = -0.020. \quad (86)$$

The interpretation would be:

- Study hours have a positive influence.
- Attendance has a positive influence.
- Previous score has a negative local influence.

Thus, the sign of the coefficient indicates the direction of the local influence.

1.15 Step 13: Why Does LIME Need Perturbed Samples?

An important question is:

Why does LIME generate perturbed samples?

The answer is that LIME needs to observe how the black-box model changes when the input changes slightly.

Consider the following:

$$x = (8, 85, 70) \tag{87}$$

with prediction

$$f(x) = 0.90. \tag{88}$$

Now reduce study hours:

$$x' = (6, 85, 70). \tag{89}$$

Suppose

$$f(x') = 0.84. \tag{90}$$

The prediction decreased:

$$0.90 - 0.84 = 0.06. \tag{91}$$

Therefore, changing study hours appears to influence the prediction.

Similarly, changing attendance gives:

$$(8, 85, 70) \rightarrow (8, 75, 70) \tag{92}$$

and

$$0.90 \rightarrow 0.78. \tag{93}$$

The decrease is

$$0.90 - 0.78 = 0.12. \tag{94}$$

This suggests that attendance has a substantial local effect in this neighborhood.

However, LIME does not simply compare one pair of observations. It generates many perturbed samples and uses weighted regression to obtain a more stable local explanation.

1.16 Step 14: Why Are Nearby Samples Given More Weight?

Suppose we have two perturbed samples:

$$d(x, z_1) = 0.50, \tag{95}$$

$$d(x, z_4) = 1.20. \tag{96}$$

Their weights are approximately:

$$\pi_x(z_1) = 0.779, \tag{97}$$

$$\pi_x(z_4) = 0.237. \tag{98}$$

Therefore,

$$\pi_x(z_1) > \pi_x(z_4). \quad (99)$$

Thus, the prediction of z_1 has greater influence when LIME learns the local model.

This is important because LIME is interested primarily in the behavior of the black-box model near the original instance.

1.17 Step 15: The Complete LIME Calculation

The entire process can now be summarized mathematically.

We begin with the original instance:

$$x = (8, 85, 70). \quad (100)$$

The black-box prediction is

$$f(x) = 0.90. \quad (101)$$

Generate perturbed samples:

$$z_1, z_2, \dots, z_n. \quad (102)$$

Obtain predictions:

$$f(z_1), f(z_2), \dots, f(z_n). \quad (103)$$

Calculate distances:

$$d(x, z_i). \quad (104)$$

Convert distances into weights:

$$\pi_x(z_i) = \exp\left(-\frac{d(x, z_i)^2}{\sigma^2}\right). \quad (105)$$

Then fit the interpretable model:

$$g(z) = \beta_0 + \beta_1 z_1 + \beta_2 z_2 + \beta_3 z_3. \quad (106)$$

The coefficients are obtained by minimizing:

$$\boxed{\text{minimize}_{g \in G} \sum_{i=1}^n \pi_x(z_i) [f(z_i) - g(z_i)]^2} \quad (107)$$

Finally, the coefficients are interpreted as local feature contributions.

1.18 Step 16: What Does “Local” Actually Mean?

This is one of the most important concepts in LIME.

Suppose that the complete machine learning model has a complicated decision boundary.

We can imagine the model as:

$$f(x) = \text{complex nonlinear function}. \quad (108)$$

It may be impossible to describe the complete function with one simple equation.

However, around the selected instance x , the behavior may be approximated using:

$$g(x) = \beta_0 + \sum_j \beta_j x_j. \quad (109)$$

Therefore,

$$\boxed{g(x) \approx f(x) \quad \text{in the neighborhood of } x.} \quad (110)$$

It does not mean that

$$g(x) \approx f(x) \quad (111)$$

for every possible input.

Therefore, LIME should not be interpreted as a global description of the machine learning model.

1.19 Step 17: Local Explanation Versus Global Explanation

Consider a complex model trained on 100,000 students.

A global explanation attempts to answer:

How does the model behave for all 100,000 students?

LIME instead asks:

Why did the model make this prediction for this particular student?

Thus,

$$\boxed{\text{Global explanation} \neq \text{LIME explanation}} \quad (112)$$

LIME provides a local explanation.

1.20 Step 18: Final LIME Explanation for the Student

The black-box model predicts:

$$\boxed{P(\text{Pass}) = 0.90} \quad (113)$$

for the student

$$x = (8, 85, 70). \quad (114)$$

LIME generates nearby samples, obtains black-box predictions, calculates proximity weights, and fits a local interpretable model.

The resulting illustrative local model is:

$$\boxed{g(x) = -0.20 + 0.040x_1 + 0.005x_2 + 0.003x_3} \quad (115)$$

The corresponding feature coefficients are:

Table 5: Final LIME explanation

Feature	Coefficient	Interpretation
Study Hours	+0.040	Strong positive influence
Attendance	+0.005	Positive influence
Previous Score	+0.003	Positive influence

Therefore, the explanation is:

The model predicts a high probability of passing for this student. In the local neighborhood of this particular student, study hours have the strongest positive influence, followed by attendance and previous examination score.

1.21 Step 19: Complete LIME Workflow

The complete workflow can be represented as:

$$\begin{array}{l} x \rightarrow \text{Perturb } x \rightarrow z_1, z_2, \dots, z_n \\ \rightarrow f(z_i) \rightarrow d(x, z_i) \rightarrow \pi_x(z_i) \\ \rightarrow \text{Weighted Regression} \rightarrow g(x) \rightarrow \text{Feature Contributions} \end{array} \quad (116)$$

In simple words:

1. Select one prediction that needs explanation.
2. Generate nearby versions of that observation.
3. Ask the black-box model to predict each nearby observation.
4. Determine how close each observation is to the original observation.
5. Give greater weight to observations that are closer.
6. Fit a simple interpretable model using these weighted observations.
7. Examine the coefficients of the simple model.
8. Use those coefficients as the local explanation.

1.22 Step 20: Important Properties of LIME

1.22.1 Model Agnostic

LIME does not require knowledge of the internal architecture of the original model.

Therefore, it can be applied to:

- Decision Trees
- Random Forests
- Support Vector Machines
- XGBoost
- Neural Networks
- Deep Learning models

The only requirement is that the model can provide predictions for input samples.

SHAP (Shapley Values): A Detailed Numerical Example

1 SHAP: Shapley Values

1.1 Introduction

SHAP stands for **SHapley Additive exPlanations**. SHAP is an explainable artificial intelligence (XAI) technique used to explain the predictions made by machine learning models.

The mathematical foundation of SHAP comes from the concept of the **Shapley value** in cooperative game theory.

The basic idea is to consider each feature as a “player” and the machine learning prediction as the “payout” of a cooperative game.

Suppose a machine learning model predicts whether a student will pass an examination.

We have three features:

- A : Study Hours
- B : Attendance
- C : Previous Examination Score

For a particular student,

$$x = (A, B, C) = (8, 85, 70). \quad (1)$$

Suppose the machine learning model predicts:

$$\boxed{f(x) = 0.90} \quad (2)$$

Thus, the model predicts a 90% probability that the student will pass.

The question is:

How much did each feature contribute to the prediction of 0.90?

SHAP answers this question by assigning a Shapley value to each feature.

1.2 1. Basic Idea of Shapley Values

The main idea is to distribute the difference between the model prediction and a baseline prediction among the features.

Suppose that when no feature information is available, the model gives the baseline prediction

$$f(\emptyset) = 0.50. \quad (3)$$

The complete model prediction is

$$f(A, B, C) = 0.90. \quad (4)$$

Therefore, the total increase from the baseline is

$$f(A, B, C) - f(\emptyset) = 0.90 - 0.50 \quad (5)$$

$$= 0.40. \quad (6)$$

Therefore, the three features together contribute

$$\boxed{0.40} \quad (7)$$

above the baseline.

The purpose of the Shapley value is to fairly distribute this 0.40 among the three features. Therefore,

$$\boxed{f(x) = f(\emptyset) + \phi_A + \phi_B + \phi_C} \quad (8)$$

where:

- $f(\emptyset)$ is the baseline prediction,
- ϕ_A is the Shapley value of Study Hours,
- ϕ_B is the Shapley value of Attendance,
- ϕ_C is the Shapley value of Previous Score.

1.3 2. Features as Players

In cooperative game theory, the entities participating in the game are called players.

In our machine learning example:

Table 1: Features considered as players

Player	Feature
A	Study Hours
B	Attendance
C	Previous Examination Score

The “game” is the prediction task.

The “payout” is the prediction produced by the machine learning model.

Therefore:

$$\boxed{\text{Features} \longrightarrow \text{Players}} \quad (9)$$

and

$$\boxed{\text{Model Prediction} \longrightarrow \text{Game Payout}} \quad (10)$$

1.4 3. All Possible Feature Coalitions

With three features, there are

$$2^3 = 8 \tag{11}$$

possible subsets, or coalitions.

They are:

$$S_1 = \emptyset, \tag{12}$$

$$S_2 = \{A\}, \tag{13}$$

$$S_3 = \{B\}, \tag{14}$$

$$S_4 = \{C\}, \tag{15}$$

$$S_5 = \{A, B\}, \tag{16}$$

$$S_6 = \{A, C\}, \tag{17}$$

$$S_7 = \{B, C\}, \tag{18}$$

$$S_8 = \{A, B, C\}. \tag{19}$$

Here, $\emptyset$ means that no feature is included.

1.5 4. Model Output for Each Coalition

To calculate Shapley values, we need the model output for every possible coalition.

For this teaching example, assume that the model produces the following outputs:

Table 2: Model output for different feature coalitions

Coalition	Model Output
$\emptyset$	0.50
$\{A\}$	0.65
$\{B\}$	0.60
$\{C\}$	0.58
$\{A, B\}$	0.75
$\{A, C\}$	0.72
$\{B, C\}$	0.68
$\{A, B, C\}$	0.90

The most important values are:

$$f(\emptyset) = 0.50 \tag{20}$$

and

$$f(A, B, C) = 0.90. \tag{21}$$

Thus,

$$0.90 - 0.50 = 0.40. \tag{22}$$

The Shapley values must add up to 0.40.

1.6 5. What is Marginal Contribution?

Before calculating Shapley values, we must understand the concept of **marginal contribution**.

The marginal contribution of feature A to a coalition S is:

$$\Delta_A(S) = f(S \cup \{A\}) - f(S). \quad (23)$$

For example, suppose the current coalition is

$$S = \{B\}. \quad (24)$$

The model output is

$$f(\{B\}) = 0.60. \quad (25)$$

Now add feature A .

The new coalition is

$$S \cup \{A\} = \{A, B\}. \quad (26)$$

The new model output is

$$f(\{A, B\}) = 0.75. \quad (27)$$

Therefore,

$$\Delta_A(\{B\}) = f(\{A, B\}) - f(\{B\}) \quad (28)$$

$$= 0.75 - 0.60 \quad (29)$$

$$= 0.15. \quad (30)$$

Therefore,

$$\boxed{\Delta_A(\{B\}) = 0.15} \quad (31)$$

This means that adding Study Hours to the coalition containing Attendance increases the prediction by 0.15.

1.7 6. Why Do We Need Different Orders?

The marginal contribution of a feature can change depending on which features are already present.

Consider the order:

$$A \rightarrow B \rightarrow C. \quad (32)$$

First, add A :

$$\Delta_A(\emptyset) = f(\{A\}) - f(\emptyset) \quad (33)$$

$$= 0.65 - 0.50 \quad (34)$$

$$= 0.15. \quad (35)$$

Next, add B :

$$\Delta_B(\{A\}) = f(\{A, B\}) - f(\{A\}) \quad (36)$$

$$= 0.75 - 0.65 \quad (37)$$

$$= 0.10. \quad (38)$$

Finally, add C :

$$\Delta_C(\{A, B\}) = f(\{A, B, C\}) - f(\{A, B\}) \quad (39)$$

$$= 0.90 - 0.75 \quad (40)$$

$$= 0.15. \quad (41)$$

Therefore,

$$0.15 + 0.10 + 0.15 = 0.40. \quad (42)$$

This equals the total difference between the final prediction and the baseline:

$$0.90 - 0.50 = 0.40. \quad (43)$$

However, consider a different order:

$$C \rightarrow B \rightarrow A. \quad (44)$$

The contributions become:

$$\Delta_C(\emptyset) = 0.58 - 0.50 = 0.08, \quad (45)$$

$$\Delta_B(\{C\}) = 0.68 - 0.58 = 0.10, \quad (46)$$

$$\Delta_A(\{B, C\}) = 0.90 - 0.68 = 0.22. \quad (47)$$

Again,

$$0.08 + 0.10 + 0.22 = 0.40. \quad (48)$$

Notice that the contribution of A changed:

$$\text{First ordering: } 0.15, \quad (49)$$

$$\text{Second ordering: } 0.22. \quad (50)$$

Therefore, simply using one ordering would not provide a fair attribution.

The Shapley value solves this problem by considering all possible orders.

1.8 7. All Possible Feature Orders

With three features, the number of possible orders is

$$3! = 6. \quad (51)$$

The six possible orders are:

$$P_1 : A \rightarrow B \rightarrow C, \quad (52)$$

$$P_2 : A \rightarrow C \rightarrow B, \quad (53)$$

$$P_3 : B \rightarrow A \rightarrow C, \quad (54)$$

$$P_4 : B \rightarrow C \rightarrow A, \quad (55)$$

$$P_5 : C \rightarrow A \rightarrow B, \quad (56)$$

$$P_6 : C \rightarrow B \rightarrow A. \quad (57)$$

We now calculate the marginal contribution of each feature for each ordering.

1.9 8. Ordering 1: $A \rightarrow B \rightarrow C$

First, add A :

$$\Delta_A(\emptyset) = 0.65 - 0.50 \quad (58)$$

$$= 0.15. \quad (59)$$

Next, add B :

$$\Delta_B(\{A\}) = 0.75 - 0.65 \quad (60)$$

$$= 0.10. \quad (61)$$

Finally, add C :

$$\Delta_C(\{A, B\}) = 0.90 - 0.75 \quad (62)$$

$$= 0.15. \quad (63)$$

Therefore,

$$\boxed{A = 0.15, \quad B = 0.10, \quad C = 0.15} \quad (64)$$

1.10 9. Ordering 2: $A \rightarrow C \rightarrow B$

First, add A :

$$0.65 - 0.50 = 0.15. \quad (65)$$

Next, add C :

$$0.72 - 0.65 = 0.07. \quad (66)$$

Finally, add B :

$$0.90 - 0.72 = 0.18. \quad (67)$$

Therefore,

$$\boxed{A = 0.15, \quad C = 0.07, \quad B = 0.18} \quad (68)$$

1.11 10. Ordering 3: $B \rightarrow A \rightarrow C$

First, add B :

$$0.60 - 0.50 = 0.10. \quad (69)$$

Next, add A :

$$0.75 - 0.60 = 0.15. \quad (70)$$

Finally, add C :

$$0.90 - 0.75 = 0.15. \quad (71)$$

Therefore,

$$\boxed{B = 0.10, \quad A = 0.15, \quad C = 0.15} \quad (72)$$

1.12 11. Ordering 4: $B \rightarrow C \rightarrow A$

First, add B :

$$0.60 - 0.50 = 0.10. \quad (73)$$

Next, add C :

$$0.68 - 0.60 = 0.08. \quad (74)$$

Finally, add A :

$$0.90 - 0.68 = 0.22. \quad (75)$$

Therefore,

$$\boxed{B = 0.10, \quad C = 0.08, \quad A = 0.22} \quad (76)$$

1.13 12. Ordering 5: $C \rightarrow A \rightarrow B$

First, add C :

$$0.58 - 0.50 = 0.08. \quad (77)$$

Next, add A :

$$0.72 - 0.58 = 0.14. \quad (78)$$

Finally, add B :

$$0.90 - 0.72 = 0.18. \quad (79)$$

Therefore,

$$\boxed{C = 0.08, \quad A = 0.14, \quad B = 0.18} \quad (80)$$

1.14 13. Ordering 6: $C \rightarrow B \rightarrow A$

First, add C :

$$0.58 - 0.50 = 0.08. \quad (81)$$

Next, add B :

$$0.68 - 0.58 = 0.10. \quad (82)$$

Finally, add A :

$$0.90 - 0.68 = 0.22. \quad (83)$$

Therefore,

$$\boxed{C = 0.08, \quad B = 0.10, \quad A = 0.22} \quad (84)$$

1.15 14. Summary of All Six Permutations

The calculations can now be summarized in one table.

Table 3: Marginal contributions for all feature orderings			
Order	Contribution of A	Contribution of B	Contribution of C
$A \rightarrow B \rightarrow C$	0.15	0.10	0.15
$A \rightarrow C \rightarrow B$	0.15	0.18	0.07
$B \rightarrow A \rightarrow C$	0.15	0.10	0.15
$B \rightarrow C \rightarrow A$	0.22	0.10	0.08
$C \rightarrow A \rightarrow B$	0.14	0.18	0.08
$C \rightarrow B \rightarrow A$	0.22	0.10	0.08

1.16 15. Calculate the Shapley Value of Study Hours

The six marginal contributions of A are:

$$0.15, \quad 0.15, \quad 0.15, \quad 0.22, \quad 0.14, \quad 0.22. \quad (85)$$

The Shapley value is the average:

$$\phi_A = \frac{0.15 + 0.15 + 0.15 + 0.22 + 0.14 + 0.22}{6} \quad (86)$$

$$= \frac{1.03}{6} \quad (87)$$

$$\approx 0.1717. \quad (88)$$

Therefore,

$$\boxed{\phi_A \approx 0.172} \quad (89)$$

Study Hours contribute approximately +0.172 to the prediction relative to the baseline.

1.17 16. Calculate the Shapley Value of Attendance

The six marginal contributions of B are:

$$0.10, \quad 0.18, \quad 0.10, \quad 0.10, \quad 0.18, \quad 0.10. \quad (90)$$

Therefore,

$$\phi_B = \frac{0.10 + 0.18 + 0.10 + 0.10 + 0.18 + 0.10}{6} \quad (91)$$

$$= \frac{0.76}{6} \quad (92)$$

$$\approx 0.1267. \quad (93)$$

Therefore,

$$\boxed{\phi_B \approx 0.127} \quad (94)$$

Attendance contributes approximately +0.127.

1.18 17. Calculate the Shapley Value of Previous Score

The six marginal contributions of C are:

$$0.15, \quad 0.07, \quad 0.15, \quad 0.08, \quad 0.08, \quad 0.08. \quad (95)$$

Therefore,

$$\phi_C = \frac{0.15 + 0.07 + 0.15 + 0.08 + 0.08 + 0.08}{6} \quad (96)$$

$$= \frac{0.61}{6} \quad (97)$$

$$\approx 0.1017. \quad (98)$$

Therefore,

$$\boxed{\phi_C \approx 0.102} \quad (99)$$

Previous Score contributes approximately +0.102.

1.19 18. Final Shapley Values

The final Shapley values are:

$$\boxed{\begin{array}{l} \phi_A \approx 0.172, \\ \phi_B \approx 0.127, \\ \phi_C \approx 0.102. \end{array}} \quad (100)$$

The sum is approximately:

$$\phi_A + \phi_B + \phi_C = 0.172 + 0.127 + 0.102 \quad (101)$$

$$\approx 0.401. \quad (102)$$

The small difference from 0.40 is due to rounding.

Using the unrounded values:

$$\phi_A + \phi_B + \phi_C = 0.40 \quad (103)$$

Therefore,

$$f(x) = f(\emptyset) + \phi_A + \phi_B + \phi_C \quad (104)$$

and

$$0.90 = 0.50 + 0.172 + 0.127 + 0.102 \quad (105)$$

approximately.

1.20 19. Interpretation of the Shapley Values

The baseline prediction is:

$$f(\emptyset) = 0.50. \quad (106)$$

The features then contribute:

Table 4: Final feature contributions		
Feature	Shapley Value	Interpretation
Study Hours	+0.172	Strongest positive contribution
Attendance	+0.127	Positive contribution
Previous Score	+0.102	Positive contribution

Therefore, the prediction can be interpreted as:

The model starts from a baseline probability of 0.50. Study Hours increase the prediction by approximately 0.172, Attendance increases the prediction by approximately 0.127, and Previous Score increases the prediction by approximately 0.102. Together, these contributions produce a prediction of approximately 0.90.

1.21 20. General Shapley Value Formula

For a model with M features, the Shapley value of feature j is:

$$\phi_j = \sum_{S \subseteq N \setminus \{j\}} \frac{|S|!(M - |S| - 1)!}{M!} [f(S \cup \{j\}) - f(S)] \quad (107)$$

where:

- N is the set of all features.
- S is a subset of features that does not contain feature j .
- $|S|$ is the number of features in S .
- $f(S)$ is the model output using coalition S .
- $f(S \cup \{j\})$ is the model output after adding feature j .
- $f(S \cup \{j\}) - f(S)$ is the marginal contribution.

The factor

$$\frac{|S|!(M - |S| - 1)!}{M!} \quad (108)$$

is the Shapley weighting factor.

1.22 21. Direct Calculation of ϕ_A Using the Shapley Formula

For our example,

$$M = 3. \quad (109)$$

For feature A , the subsets that do not contain A are:

$$\emptyset, \quad \{B\}, \quad \{C\}, \quad \{B, C\}. \quad (110)$$

Therefore,

$$\phi_A = \frac{0!2!}{3!} [f(\{A\}) - f(\emptyset)] \quad (111)$$

$$+ \frac{1!1!}{3!} [f(\{A, B\}) - f(\{B\})] \quad (112)$$

$$+ \frac{1!1!}{3!} [f(\{A, C\}) - f(\{C\})] \quad (113)$$

$$+ \frac{2!0!}{3!} [f(\{A, B, C\}) - f(\{B, C\})]. \quad (114)$$

The weights are:

$$\frac{0!2!}{3!} = \frac{1}{3}, \quad (115)$$

$$\frac{1!1!}{3!} = \frac{1}{6}, \quad (116)$$

$$\frac{2!0!}{3!} = \frac{1}{3}. \quad (117)$$

Substituting the model outputs:

$$\phi_A = \frac{1}{3}(0.65 - 0.50) + \frac{1}{6}(0.75 - 0.60) \quad (118)$$

$$+ \frac{1}{6}(0.72 - 0.58) + \frac{1}{3}(0.90 - 0.68). \quad (119)$$

Therefore,

$$\phi_A = \frac{1}{3}(0.15) + \frac{1}{6}(0.15) + \frac{1}{6}(0.14) + \frac{1}{3}(0.22) \quad (120)$$

$$= 0.0500 + 0.0250 + 0.0233 + 0.0733 \quad (121)$$

$$\approx 0.1717. \quad (122)$$

Thus,

$$\boxed{\phi_A \approx 0.172} \quad (123)$$

This is exactly the same result obtained by averaging the marginal contributions over all six feature orderings.

1.23 22. Shapley Weights for Three Features

For three features, the possible coalition sizes are 0, 1, and 2.

The corresponding Shapley weights are:

Table 5: Shapley weights for three features

Coalition Size $ S $	Shapley Weight
0	$\frac{1}{3}$
1	$\frac{1}{6}$
2	$\frac{1}{3}$

Therefore, students can understand the Shapley value as:

$$\boxed{\text{Shapley Value} = \text{Weighted Average of Marginal Contributions}} \quad (124)$$

1.24 23. Important Property: Efficiency

One of the important properties of Shapley values is **efficiency**.

The sum of all feature contributions must equal the difference between the final prediction and the baseline:

$$\boxed{\sum_{j=1}^M \phi_j = f(x) - f(\emptyset)} \quad (125)$$

For our example:

$$\phi_A + \phi_B + \phi_C = 0.40 \quad (126)$$

$$= 0.90 - 0.50. \quad (127)$$

Therefore,

$$\boxed{0.90 = 0.50 + \phi_A + \phi_B + \phi_C} \quad (128)$$

This is why Shapley values provide an additive explanation of the prediction.

1.25 24. Positive and Negative Shapley Values

A Shapley value can be positive or negative.

If

$$\phi_j > 0, \quad (129)$$

then feature j pushes the prediction above the baseline.

If

$$\phi_j < 0, \quad (130)$$

then feature j pushes the prediction below the baseline.

For example, suppose another student has:

$$\phi_A = 0.15, \quad \phi_B = -0.08, \quad \phi_C = 0.03. \quad (131)$$

Then:

- Study Hours increase the prediction.
- Attendance decreases the prediction.
- Previous Score slightly increases the prediction.

The final prediction would be:

$$f(x) = f(\emptyset) + 0.15 - 0.08 + 0.03. \quad (132)$$

Thus, a negative Shapley value is not an error. It simply means that the feature moves the prediction in the negative direction relative to the baseline.

1.26 25. SHAP Explanation in Simple Words

The complete process can be summarized as:

Start with baseline prediction ↓ Add features in different orders ↓ Calculate marginal contribution ↓ Average contributions over all orders ↓ Obtain Shapley values	(133)
---	-------

For our example:

$\text{Prediction} = \text{Baseline} + \text{Study Hours} + \text{Attendance} + \text{Previous Score}$	(134)
--	-------

Numerically:

$0.90 = 0.50 + 0.172 + 0.127 + 0.102$	(135)
---------------------------------------	-------

approximately.

1.27 26. SHAP Versus LIME

Both SHAP and LIME are popular XAI techniques, but their approaches are different.

LIME can be summarized as:

$\text{Perturb} \rightarrow \text{Predict} \rightarrow \text{Weight} \rightarrow \text{Fit Simple Model}$	(136)
---	-------

SHAP can be summarized as:

$\text{Generate Coalitions} \rightarrow \text{Calculate Marginal Contributions} \rightarrow \text{Average}$	(137)
---	-------

Table 6: Comparison between LIME and SHAP

Property	LIME	SHAP
Basic idea	Fits a local interpretable surrogate model	Assigns Shapley values to features
Foundation	Local surrogate modeling	Cooperative game theory
Main calculation	Weighted regression	Weighted marginal contributions
Feature contribution	Obtained from surrogate-model coefficients	Obtained from Shapley values
Interpretation	Local approximation	Additive feature attribution

1.28 27. Final Interpretation for Students

For the selected student:

$$x = (8, 85, 70), \quad (138)$$

the model predicts:

$$f(x) = 0.90. \quad (139)$$

The baseline prediction is:

$$f(\emptyset) = 0.50. \quad (140)$$

The Shapley values are approximately:

$$\phi_A = +0.172, \quad (141)$$

$$\phi_B = +0.127, \quad (142)$$

$$\phi_C = +0.102. \quad (143)$$

Therefore:

$$\boxed{0.90 \approx 0.50 + 0.172 + 0.127 + 0.102} \quad (144)$$

The interpretation is:

The model starts with a baseline probability of 50%. Study Hours contribute approximately 17.2 percentage points, Attendance contributes approximately 12.7 percentage points, and Previous Score contributes approximately 10.2 percentage points. Together, these contributions increase the prediction to approximately 90%.

1.29 28. Key Points to Remember

Students should remember the following:

1. Each feature is treated as a player in a cooperative game.
2. A coalition is a subset of features.
3. The marginal contribution measures how much the prediction changes when a feature is added.

4. The contribution of a feature can depend on which features are already present.
5. Shapley values consider all possible feature orderings or, equivalently, all possible coalitions.
6. The Shapley value is a weighted average of marginal contributions.
7. Positive Shapley values increase the prediction relative to the baseline.
8. Negative Shapley values decrease the prediction relative to the baseline.
9. The sum of all Shapley values equals the difference between the prediction and the baseline.

The most important equation is:

$$\phi_j = \sum_{S \subseteq N \setminus \{j\}} \frac{|S|!(M - |S| - 1)!}{M!} [f(S \cup \{j\}) - f(S)] \quad (145)$$

The most important interpretation is:

$$\text{Prediction} = \text{Baseline} + \text{Feature Contributions} \quad (146)$$

For this example:

$$\text{Prediction} = 0.50 + 0.172 + 0.127 + 0.102 \approx 0.90 \quad (147)$$

Therefore, SHAP answers the question:

“How much did each feature contribute to this prediction?”